

Anthos 服務網格工作坊：實驗室指南

來源：<https://codelabs.developers.google.com/codelabs/anthos-service-mesh-workshop?hl=zh-tw>

步驟數：17

本研討會提供實作體驗，逐步說明如何在 GCP 的實際工作環境中設定遍及全球的服務。主要用於運算和 Anthos 服務網格的 GKE 技術，用於建立安全連線、觀測能力和進階流量型態。本工作坊採用的所有做法和工具，就是會用於實際工作環境的。

目錄

1. [1. ALPHA WORKSHOP](#)
2. [2. 總覽](#)
3. [3. 基礎架構設定 - 管理員工作流程](#)
4. [4. 實驗室設定和準備](#)
5. [5. 基礎架構設定 - 使用者工作流程](#)
6. [6. 基礎架構存放區說明](#)
7. [7. 部署範例應用程式](#)
8. [8. 使用 Stackdriver 進行觀測](#)
9. [9. 雙向傳輸層安全標準 \(TLS\) 驗證](#)
10. [10. 初期測試部署](#)
11. [11. 授權政策](#)
12. [12. 基礎架構資源調度](#)
13. [13. 斷路機制](#)
14. [14. 錯誤注入](#)
15. [15. 監控 Istio 控制層](#)
16. [16. 排解 Istio 問題](#)
17. [17. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. ALPHA WORKSHOP

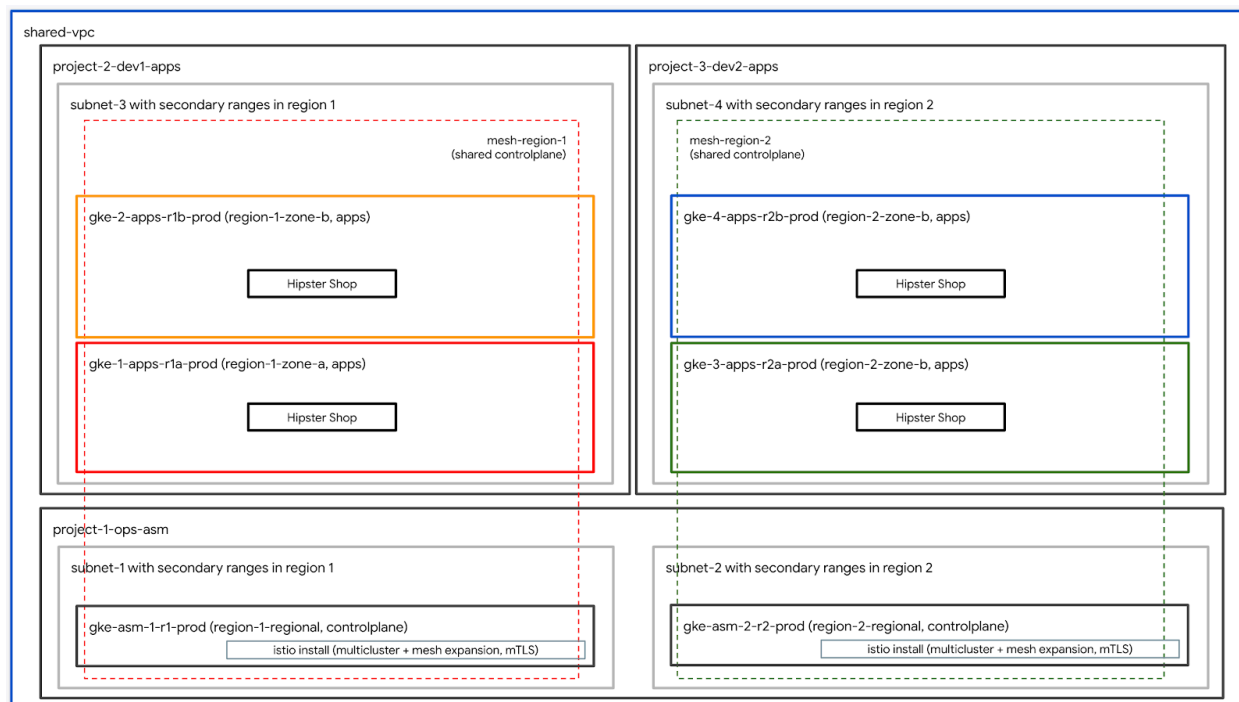
重要事項：這是 Alpha 版研討會，僅供講師帶領學員進行課堂訓練。

工作坊程式碼研究室連結：bit.ly/asm-workshop

步驟 2 · 約 0 分鐘

2. 總覽

架構圖



本研討會提供身歷其境的實作體驗，說明如何在 GCP 上設定實際工作環境中分散在全球各地的服務。主要使用的技術包括：[Google Kubernetes Engine \(GKE\)](#) 用於運算，以及 [Istio 服務網格](#)，用於建立安全連線、可觀測性及進階流量控管。本研討會使用的所有做法和工具，都與您在實際製作時使用的相同。

簡報大綱

- 單元 0 - 簡介和平台設定
- 簡介與架構
- 服務網格和 Istio/ASM 簡介
- 實驗室：基礎架構設定：使用者工作流程
- 休息時間
- QnA
- 單元 1 - 使用 ASM 安裝、保護及監控應用程式
- 存放區模型：基礎架構和 Kubernetes 存放區說明
- 實驗室：部署範例應用程式
- 分散式服務和可觀測性
- 午餐
- 實驗室：使用 Stackdriver 進行可觀測性
- QNA
- 模組 2 - DevOps - Canary 推出、政策/RBAC
- 多叢集服務探索和安全性/政策
- 實驗室：雙向傳輸層安全標準 (TLS)
- 初期測試部署

- **實驗室：初期測試部署**
- 安全的多叢集全域負載平衡
- 休息時間
- **實驗室：授權政策**
- **QNA**
- **模組 3 - 基礎架構營運 - 平台升級**
- 分散式服務建構模塊
- **實驗室：基礎架構擴充**
- 後續步驟

簡報

如要查看本次研討會的投影片，請點選以下連結：

[ASM 工作坊投影片](#)

必要條件

繼續進行本研討會前，請先完成下列事項：

1. [GCP 機構](#)節點
 2. [帳單帳戶 ID](#) (使用者必須是這個帳單帳戶的[帳單管理員](#))
 3. 使用者在機構層級的[機構管理員 IAM 角色](#)
-

3. 基礎架構設定 - 管理員工作流程

本節中的步驟由建立研討會的管理員執行。

如果您是參加研討會的使用者，請跳至下一個標題為 `Infrastructure Setup - User Workflow` 的章節

說明啟動工作坊指令碼

名為 `bootstrap_workshop.sh` 的指令碼用於設定工作坊的初始環境。如果您要為多位使用者提供訓練，可以使用這個指令碼為自己設定單一環境，或為多位使用者設定多個環境。

自我啟動工作坊指令碼需要下列輸入內容：

- **機構名稱** (例如 `yourcompany.com`)：您要在這個機構中建立研討會環境。
- **帳單 ID** (例如 `12345-12345-12345`) - 這個帳單 ID 用於支付研討會期間使用的所有資源。
- **工作坊編號** (例如 `01`) - 兩位數的數字。如果您在同一天舉辦多場研討會，並想分別追蹤這些研討會，這項功能就非常實用。專案 ID 也是從研討會編號衍生而來。這樣一來，您每次都能取得不重複的專案 ID。除了研討會編號，專案 ID 也會使用當天日期 (格式為 `YYMMDD`)。日期和研討會編號的組合會產生不重複的專案 ID。
- **起始使用者編號** (例如 `1`) - 這個編號代表研討會中的第一位使用者。舉例來說，如要為 10 位使用者建立研討會，開始使用者編號可能是 1，結束使用者編號可能是 10。
- **使用者編號** (例如 `10`) - 這個數字代表研討會的最後一位使用者。舉例來說，如要為 10 位使用者建立研討會，開始使用者編號可能是 1，結束使用者編號可能是 10。如果您要設定單一環境 (例如供自己使用)，請將開始和結束使用者人數設為相同。這會建立單一環境。

除了日期和研討會編號，專案 ID 也會納入開始和結束使用者編號。使用者 ID (適用於執行研討會的使用者) 是從開始和結束使用者號碼衍生而來。使用者 ID 的說明請參閱「`User schema and permissions performing the workshop`」一節。

- **管理員 GCS bucket** (例如 `my-gcs-bucket-name`) - GCS bucket 用於儲存研討會相關資訊。`cleanup_workshop.sh` 指令碼會使用這項資訊，順利刪除在啟動研討會指令碼期間建立的所有資源。建立研討會的管理員必須具備這個 bucket 的讀取/寫入權限。

啟動程序工作坊指令碼會使用上述值，並做為呼叫 `setup-terraform-admin-project.sh` 指令碼的包裝函式指令碼。`setup-terraform-admin-project.sh` 指令碼會為單一使用者建立研討會環境。

啟動研討會需要管理員權限

本研討會的使用者分為兩種類型。`ADMIN_USER`，負責建立及刪除本研討會的資源。第二位是 `MY_USER`，負責執行研討會中的步驟。`MY_USER` 只能存取自己的資源。「`ADMIN_USER`」可存取所有使用者設定。如果您是為自己建立這項設定，則 `ADMIN_USER` 和 `MY_USER` 相同。如果您是為多位學生建立這項研討會的老師，則 `ADMIN_USER` 和 `MY_USER` 會有所不同。

如要使用 `ADMIN_USER`，必須具備下列組織層級權限：

- **擁有者**：擁有機構中所有專案的專案擁有者權限。
- **資料夾管理員**：可在機構中建立及刪除資料夾。每位使用者都會獲得一個資料夾，內含專案中的所有資源。
- **機構管理員**

- **專案建立者**：可在機構中建立專案。
- **專案刪除者** - 能夠刪除機構中的專案。
- **專案 IAM 管理員** - 能夠在機構中的所有專案建立 IAM 規則。

此外，`ADMIN_USER` 也必須是研討會所用帳單 ID 的帳單管理員。

執行研討會的使用者結構和權限

如果您打算為機構中的使用者 (而非自己) 建立這項研討會，請務必遵循 `MY_USERS` 的特定使用者命名方式。在 `bootstrap_workshop.sh` 指令碼中，您會提供起始和結束使用者編號。系統會使用這些號碼建立下列使用者名稱：

- `user<3 digit user number>@<organization_name>`

舉例來說，如果您執行啟動工作坊指令碼，起始使用者編號為 1，結束使用者編號為 3，且貴機構名為 `yourcompany.com`，系統會為下列使用者建立工作坊環境：

- `user001@yourcompany.com`
- `user002@yourcompany.com`
- `user003@yourcompany.com`

這些使用者名稱會獲派專案擁有者角色，以管理在 `setup_terraform_admin_project.sh` 指令碼執行期間建立的特定專案。使用啟動程序指令碼時，您必須遵守這個使用者命名架構。請參閱[這篇文章](#)，瞭解如何在 G Suite 中一次新增多位使用者。

工作坊所需工具

本研討會旨在從 Cloud Shell 啟動。本研討會需要下列工具。

- [gcloud](#) (版本 `>= 270`)
- [kubectl](#)
- [sed](#) (適用於 Cloud Shell/Linux 上的 `sed`，不適用於 Mac OS)
- [git](#) (請確認您使用的是最新版本)
- `sudo apt update`
- `sudo apt install git`
- [jq](#)
- [envsubst](#)
- [kustomize](#)

Cloud Shell 預設已安裝所有工具，但 `kustomize` 除外。如果尚未安裝 `kustomize`，請使用上述連結提供的 Bash 指令碼安裝，並確保 `kustomize` 二進位檔位於 `PATH` 中，然後在 Cloud Shell 中執行 `kustomize` 指令來確認。

為自己設定工作坊 (單一使用者設定)

1. 開啟 Cloud Shell，並在 Cloud Shell 中執行下列所有動作。點選下方連結。

Cloud Shell

2. 確認您已使用預期的管理員使用者登入 `gcloud`。

```
gcloud config list
```

3. 建立 `WORKDIR` 並複製研討會存放區。

```
mkdir asm-workshop
cd asm-workshop
export WORKDIR=`pwd`
git clone https://github.com/GoogleCloudPlatform/anthos-service-mesh-workshop.git asm
```

4. 定義機構名稱、帳單 ID、研討會編號和管理員 GCS bucket，以用於研討會。請參閱上方的各節，瞭解設定研討會所需的權限。

```
gcloud organizations list
export ORGANIZATION_NAME=<ORGANIZATION NAME>

gcloud beta billing accounts list
export ADMIN_BILLING_ID=<ADMIN_BILLING ID>

export WORKSHOP_NUMBER=<two digit number for example 01>

export ADMIN_STORAGE_BUCKET=<ADMIN CLOUD STORAGE BUCKET>
```

5. 執行 `bootstrap_workshop.sh` 指令碼。這個指令碼可能需要幾分鐘才能完成。

```
cd asm
./scripts/bootstrap_workshop.sh --org-name ${ORGANIZATION_NAME} --billing-id
${ADMIN_BILLING_ID} --workshop-num ${WORKSHOP_NUMBER} --admin-gcs-bucket
${ADMIN_STORAGE_BUCKET} --set-up-for-admin
```

請注意，由於您是自行設定，因此不必定義開始和結束使用者人數。

`bootstrap_workshop.sh` 指令碼完成後，系統會為機構內的每位使用者建立 GCP 資料夾。系統會在資料夾中建立 [Terraform 管理員專案](#)。Terraform 管理員專案用於建立本研討會所需的其餘 GCP 資源。在 Terraform 管理員專案中啟用必要的 API。您可以使用 [Cloud Build](#) 套用 Terraform 方案。請為 [Cloud Build 服務帳戶](#) 授予適當的 IAM 角色，讓該帳戶能夠在 GCP 上建立資源。最後，您會在 [Google Cloud Storage \(GCS\)](#) 值區中設定遠端後端，以儲存所有 GCP 資源的 [Terraform 狀態](#)。

如要在 Terraform 管理員專案中查看 Cloud Build 工作，您需要 Terraform 管理員專案 ID。這項資訊會儲存在 `asm` 目錄下的 `vars/vars.sh` 檔案中。如果您以管理員身分自行設定研討會，這個目錄才會保留。

6. 為變數檔案設定來源，以設定環境變數


```
echo "export WORKDIR=$WORKDIR" >> $WORKDIR/asm/vars/vars.sh
source $WORKDIR/asm/vars/vars.sh
```

為多位使用者設定工作坊 (多使用者設定)

7. 開啟 Cloud Shell，並在 Cloud Shell 中執行下列所有動作。點選下方連結。

Cloud Shell

8. 確認您已使用預期的管理員使用者登入 gcloud。

```
gcloud config list
```

9. 建立 `WORKDIR` 並複製研討會存放區。

```
mkdir asm-workshop
cd asm-workshop
export WORKDIR=`pwd`
git clone https://github.com/GoogleCloudPlatform/anthos-service-mesh-workshop.git asm
```

10. 定義機構名稱、帳單 ID、研討會編號、開始和結束使用者人數，以及用於研討會的管理員 GCS bucket。請參閱上方的各節，瞭解設定研討會所需的權限。

```
gcloud organizations list
export ORGANIZATION_NAME=<ORGANIZATION NAME>

gcloud beta billing accounts list
export ADMIN_BILLING_ID=<BILLING ID>

export WORKSHOP_NUMBER=<two digit number for example 01>

export START_USER_NUMBER=<number for example 1>

export END_USER_NUMBER=<number greater or equal to START_USER_NUM>

export ADMIN_STORAGE_BUCKET=<ADMIN CLOUD STORAGE BUCKET>
```

11. 執行 `bootstrap_workshop.sh` 指令碼。這個指令碼可能需要幾分鐘才能完成。

```
cd asm
./scripts/bootstrap_workshop.sh --org-name ${ORGANIZATION_NAME} --billing-id
${ADMIN_BILLING_ID} --workshop-num ${WORKSHOP_NUMBER} --start-user-num ${START_USER_NUMBER} -
--end-user-num ${END_USER_NUMBER} --admin-gcs-bucket ${ADMIN_STORAGE_BUCKET}
```

12. 從管理員 GCS bucket 取得 workshop.txt 檔案，以擷取 Terraform 專案 ID。

```
export WORKSHOP_ID="$(date '+%Y%m%d')-${WORKSHOP_NUMBER}"  
gsutil cp gs://${ADMIN_STORAGE_BUCKET}/${ORGANIZATION_NAME}/${WORKSHOP_ID}/workshop.txt .
```

4. 實驗室設定和準備

使用者可從這裡開始進行研討會。在 Chrome 或 Firefox 的 `INCOGNITO` 視窗中完成所有步驟。

選擇實驗室路徑

本研討會的實驗室可透過下列兩種方式執行：

- 「簡單快速的互動式指令碼」方式
- 「手動複製並貼上每項指令」的方式

快速入門指令碼方法可讓您為每個實驗室執行單一互動式指令碼，自動執行該實驗室的指令，引導您完成實驗室。這些指令會分批執行，並簡要說明每個步驟和完成的動作。每批指令執行完畢後，系統會提示你繼續執行下一批指令。這樣一來，您就能按照自己的步調完成實驗室。快速通道指令碼是**冪等**的，也就是說，您可以多次執行這些指令碼，結果都相同。

快速入門指令碼會顯示在每個實驗室頂端的綠色方塊中，如下所示。

快速入門指令碼

```
${WORKDIR}/asm/fasttrack_scripts/<name_of_script>.sh
```

複製及貼上方法是傳統做法，也就是複製及貼上個別指令區塊，並附上指令說明。這個方法只能執行一次。我們無法保證以這個方法重新執行指令會得到相同結果。

進行實驗室時，請選擇其中一種方法。

選擇方法並完成相關步驟後，請勿執行其他方法中的步驟。

快速設定指令碼

完成快速入門指令碼後，您就完成了該實驗室。請勿執行「複製並貼上實驗室操作說明」中的步驟。

取得使用者資訊

本研討會將使用研討會管理員建立的臨時使用者帳戶 (或實驗室帳戶) 進行。實驗室帳戶擁有研討會中的所有專案。研討會管理員會提供實驗室帳戶憑證 (使用者名稱和密碼) 給研討會使用者。所有使用者專案都會加上實驗室帳戶的使用者名稱做為前置字元，例如實驗室帳戶 `user001@yourcompany.com` 的 Terraform 管理員專案 ID 為 `user001-200131-01-tf-abcde`，其餘專案也依此類推。每位使用者都必須使用研討會管理員提供的實驗室帳戶登入，並使用該帳戶進行研討會。

請檢查您的公司電子郵件地址 (即您用來報名研討會的電子郵件地址)。您應該已收到研討會管理員的電子郵件，內含實驗室帳戶的憑證 (即使用者名稱和密碼)。電子郵件內容如下所示。

Hello

Thank you for your interest in attending the ASM Workshop. Your user details for the workshop are as follows:

Username: user311@gcpworkshops.com

Password: yoX0\$PcQ6xyz|

We look forward to seeing you at the workshop

Thanks.

GCP Workshops Admin.

1. 點選下方連結開啟 Cloud Shell。

Cloud Shell

2. 使用實驗室帳戶憑證登入 (請勿使用公司或個人帳戶登入)。實驗室帳戶看起來像

`userXYZ@<workshop_domain>.com`。



Sign in

Use your Google Account

Email or phone

`user001@gcpworkshops.dev|`

3. 由於這是新帳戶，系統會提示您接受《[Google 服務條款](#)》。按一下「接受」。

Welcome to your new account

Welcome to your new account: user001@gcpworkshops.dev. Your account is compatible with many [Google services](#), but your gcpworkshops.dev administrator decides which services you may access using your account. For tips about using your new account, visit the [Google Help Center](#).

When you use Google services, your domain administrator will have access to your user001@gcpworkshops.dev account information, including any data you store with this account in Google services. You can learn more [here](#), or by consulting your organization's privacy policy, if one exists. You can choose to maintain a separate account for your personal use of any Google services, including email. If you have multiple Google accounts, you can [manage which account you use](#) with Google services and [switch between them](#) whenever you choose. Your username and profile picture can help you ensure that you're using the intended account.

If your organization provides you access to the G Suite [core services](#), your use of those services is governed by your organization's G Suite agreement. Any other Google services your administrator enables ("Additional Services") are available to you under the [Google Terms of Service](#) and the [Google Privacy Policy](#). Certain Additional Services may also have [service-specific terms](#). Your use of any services your administrator allows you to access constitutes acceptance of applicable service-specific terms.

Click "Accept" below to indicate that you understand this description of how your user001@gcpworkshops.dev account works and agree to the [Google Terms of Service](#) and the [Google Privacy Policy](#).

Accept

4. 在下一個畫面中，勾選核取方塊來同意《[Google 服務條款](#)》，然後按一下 **Start Cloud Shell**。

 **Cloud Shell**

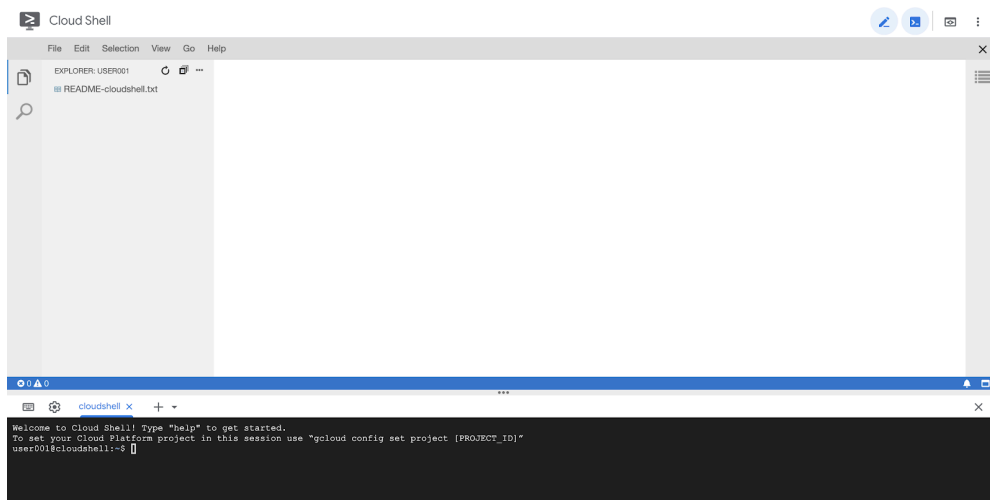
Manage your projects and resources from your web browser with Google Cloud Shell, an interactive shell environment.

With Cloud Shell, the Cloud SDK gcloud command-line tool and other utilities are pre-installed, fully authenticated, and up-to-date. [Learn more](#).

☒ I agree that my use of any Google Cloud Platform Services is subject to my compliance with the applicable [Google Cloud Platform Terms of Service](#).

Start Cloud Shell

這個步驟會為您佈建小型 Linux Debian VM，供您存取 GCP 資源。每個帳戶都會取得 Cloud Shell VM。使用實驗室帳戶登入，系統會提供實驗室帳戶憑證並登入。除了 Cloud Shell，系統也會佈建程式碼編輯器，方便您編輯設定檔 (Terraform、YAML 等)。根據預設，Cloud Shell 畫面會分割為 Cloud Shell 環境 (底部) 和 Cloud Code 編輯器 (頂端)。



右上角的鉛筆



和殼層提示



圖示可供切換殼層和程式碼編輯器。你也可以拖曳中間的分隔線 (向上或向下)，手動變更每個視窗的大小。5. 為本研討會建立 WORKDIR。WORKDIR 是您執行本研討會所有實驗室的資料夾。在 Cloud Shell 中執行下列指令，建立 WORKDIR。

```
mkdir -p ${HOME}/asm-workshop
cd ${HOME}/asm-workshop
export WORKDIR=`pwd`
```

6. 將實驗室帳戶使用者匯出為變數，以用於本研討會。這個帳戶與您登入 Cloud Shell 時使用的帳戶相同。

```
export MY_USER=<LAB ACCOUNT EMAIL PROVIDED BY THE WORKSHOP ADMIN>
# For example export MY_USER=user001@gcpworkshops.com
```

注意：如果您是管理員，並使用「-sufa」或「-setup-for-admin」旗標為自己設定研討會，請將 MY_USER 設為您用來建立研討會專案的 GCP 帳戶電子郵件地址。

7. 執行下列指令，回應 WORKDIR 和 MY_USER 變數，確保兩者都設定正確。

```
echo "WORKDIR set to ${WORKDIR}" && echo "MY_USER set to ${MY_USER}"
```

8. 複製研討會存放區。

```
git clone https://github.com/GoogleCloudPlatform/anthos-service-mesh-workshop.git
${WORKDIR}/asm
```

5. 基礎架構設定 - 使用者工作流程

目標：驗證基礎架構和 Istio 安裝作業

- 安裝工作坊工具
- 複製工作坊存放區
- 驗證 `Infrastructure` 安裝
- 驗證 `k8s-repo` 安裝
- 驗證 Istio 安裝

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/infrastructure-setup-user-workflow.sh
```

指令碼執行完成後，請提供 `bashrc` 來源，更新 Cloud Shell 工作階段中的變數。

```
source ${HOME}/.bashrc
```

複製及貼上方法實驗室操作說明

重要事項：如果您已完成本實驗室的快速通道指令碼，請勿執行下方的複製及貼上方法。

取得使用者資訊

設定研討會的管理員必須向使用者提供使用者名稱和密碼資訊。所有使用者專案都會加上使用者名稱前置字元，例如使用者 `user001@yourcompany.com` 的 Terraform 管理員專案 ID 為 `user001-200131-01-tf-abcde`，其餘專案依此類推。每位使用者只能存取自己的研討會環境。

工作坊所需工具

本研討會旨在從 Cloud Shell 啟動。本研討會需要下列工具。

Cloud Shell 中已預設安裝所有工具，但 `kustomize` 和 `pv` 除外。下方將說明如何安裝這兩項工具。

- `gcloud` (版本 `>= 270`)
- `kubectl`
- `sed` (適用於 Cloud Shell/Linux 上的 `sed`，不適用於 Mac OS)
- `git` (請確認您使用的是最新版本)
- `sudo apt update`
- `sudo apt install git`
- `jq`
- `envsubst`

- [kustomize](#)
- [pv](#)

存取 Terraform 管理員專案

`bootstrap_workshop.sh` 指令碼完成後，系統會為機構內的每位使用者建立 GCP 資料夾。系統會在資料夾中建立 [Terraform 管理員專案](#)。Terraform 管理員專案用於建立本研討會所需的其餘 GCP 資源。`setup-terraform-admin-project.sh` 指令碼會在 Terraform 管理員專案中啟用必要的 API。[Cloud Build](#) 用於套用 Terraform 方案。透過指令碼，您可為 [Cloud Build 服務帳戶](#) 指派適當的 IAM 角色，讓該帳戶能夠在 GCP 上建立資源。最後，在 [Google Cloud Storage \(GCS\)](#) bucket 中設定遠端後端，以儲存所有 GCP 資源的 [Terraform 狀態](#)。

如要在 Terraform 管理員專案中查看 Cloud Build 工作，您需要 Terraform 管理員專案 ID。這會儲存在啟動指令碼中指定的管理員 GCS bucket。如果您為多位使用者執行啟動程序指令碼，所有 Terraform 管理員專案 ID 都會位於 GCS 值區中。

本研討會的練習將在 Cloud Shell 和 Cloud 控制台中進行。由於您將使用不同的使用者名稱和密碼登入，請在 [INCOGNITO](#) 視窗中執行下列步驟。

1. 點選下方連結，開啟 Cloud Shell (如果尚未在「實驗室設定和準備」部分開啟)。

Cloud Shell

2. 在 `$HOME/bin` 資料夾中安裝 `kustomize` (如果尚未安裝)，並將 `$HOME/bin` 資料夾新增至 `$PATH`。

```
mkdir -p $HOME/bin
cd $HOME/bin
curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
cd $HOME
export PATH=$PATH:${HOME}/bin
echo "export PATH=$PATH:$HOME/bin" >> $HOME/.bashrc
```

3. 安裝 `pv` 並移至 `$HOME/bin/pv`。

```
sudo apt-get update && sudo apt-get -y install pv
sudo mv /usr/bin/pv ${HOME}/bin/pv
```

4. 更新 Bash 提示。

```
cp $WORKDIR/asm/scripts/krompt.bash $HOME/.krompt.bash
echo "export PATH=\$PATH:\$HOME/bin" >> $HOME/.asm-workshop.bash
echo "source $HOME/.krompt.bash" >> $HOME/.asm-workshop.bash

alias asm-init='source $HOME/.asm-workshop.bash' >> $HOME/.bashrc
echo "source $HOME/.asm-workshop.bash" >> $HOME/.bashrc
source $HOME/.bashrc
```


5. 確認您已使用預期使用者帳戶登入 gcloud。

```
echo "Check logged in user output from the next command is $MY_USER"
gcloud config list account --format=json | jq -r .core.account
```

6. 執行下列指令，取得 Terraform 管理員專案 ID：

```
export TF_ADMIN=$(gcloud projects list | grep tf- | awk '{ print $1 }')
echo $TF_ADMIN
```

7. 與研討會相關的所有資源都會以變數的形式，儲存在 Terraform 管理員專案的 GCS 值區中，vars.sh 檔案內。取得 Terraform 管理員專案的 vars.sh 檔案。

```
mkdir $WORKDIR/asm/vars
gsutil cp gs://$TF_ADMIN/vars/vars.sh $WORKDIR/asm/vars/vars.sh
echo "export WORKDIR=$WORKDIR" >> $WORKDIR/asm/vars/vars.sh
```

8. 按一下顯示的連結，開啟 Terraform 管理專案的 Cloud Build 頁面，並確認建構作業已順利完成。

```
source $WORKDIR/asm/vars/vars.sh
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_ADMIN}"
```

如果是首次存取 Cloud Console，請同意 Google 服務條款。

9. 現在您已進入 **Cloud Build** 頁面，請按一下左側導覽列中的 **History** 連結，然後按一下最新版本，查看初始 Terraform 應用程式的詳細資料。下列資源是透過 Terraform 指令碼建立。您也可以參考上方的架構圖。

- 機構中的 4 個 GCP 專案。系統會將您提供的帳單帳戶與每個專案建立關聯。
- 其中一個專案是共用 VPC 的 **network host project**。這項專案中不會建立其他資源。
- 其中一個專案是 **ops project**，用於 Istio 控制層 GKE 叢集。
- 這兩個專案代表兩個不同的開發團隊，分別負責各自的服務。
- 在 **ops**、**dev1** 和 **dev2** 這三個專案中，各建立兩個 GKE 叢集。
- 系統會建立名為 **k8s-repo** 的 CSR 存放區，其中包含六個 Kubernetes 資訊清單檔案的資料夾。每個 GKE 叢集各有一個資料夾。這個存放區用於以 GitOps 方式將 Kubernetes 資訊清單部署至叢集。
- 系統會建立 **Cloud Build 觸發條件**，因此只要 **k8s-repo** 的主要分支有任何提交內容，就會將 Kubernetes 資訊清單從各自的資料夾部署至 GKE 叢集。

Terraform 指令碼需要 40 至 50 分鐘才能完成。

10. 建構完成後，**terraform admin project** 另一個建構作業會在 **ops** 專案中啟動。點選顯示的連結，開啟 **ops project** 的 Cloud Build 頁面，並確認 k8s-repo Cloud Build 已順利完成。

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

注意：您必須先完成這項建構作業，才能驗證 Istio 安裝作業。

驗證安裝

11. 為所有叢集建立 `kubeconfig` 檔案。執行下列指令碼。

```
$WORKDIR/asm/scripts/setup-gke-vars-kubeconfig.sh
```

這個指令碼會在 `gke` 資料夾中建立名為 `kubemesh` 的新 `kubeconfig` 檔案。

12. 變更 `KUBECONFIG` 變數，指向新的 `kubeconfig` 檔案。

```
source $WORKDIR/asm/vars/vars.sh
export KUBECONFIG=$WORKDIR/asm/gke/kubemesh
```

13. 將 `vars.sh` 和 `KUBECONFIG` 變數新增至 Cloud Shell 的 `.bashrc`，這樣每次重新啟動 Cloud Shell 時，系統都會提供來源。

```
echo "source ${WORKDIR}/asm/vars/vars.sh" >> $HOME/.bashrc
echo "export KUBECONFIG=${WORKDIR}/asm/gke/kubemesh" >> $HOME/.bashrc
```

14. 列出叢集環境。您應該會看到六個叢集。

```
kubectl config view -ojson | jq -r '.clusters[].name'
```

``Output (do not copy)``

```
gke_tf05-01-ops_us-central1_gke-asm-2-r2-prod
gke_tf05-01-ops_us-west1_gke-asm-1-r1-prod
gke_tf05-02-dev1_us-west1-a_gke-1-apps-r1a-prod
gke_tf05-02-dev1_us-west1-b_gke-2-apps-r1b-prod
gke_tf05-03-dev2_us-central1-a_gke-3-apps-r2a-prod
gke_tf05-03-dev2_us-central1-b_gke-4-apps-r2b-prod
```

驗證 Istio 安裝

15. 請檢查所有 Pod 是否正在執行，以及所有工作是否已完成，確認兩個叢集都已安裝 Istio。

```
kubectl --context ${OPS_GKE_1} get pods -n istio-system
kubectl --context ${OPS_GKE_2} get pods -n istio-system
```

`Output (do not copy)`

NAME	READY	STATUS	RESTARTS	AGE
grafana-5f798469fd-z9f98	1/1	Running	0	6m21s
istio-citadel-568747d88-qdw64	1/1	Running	0	6m26s
istio-egressgateway-8f454cf58-ckw7n	1/1	Running	0	6m25s
istio-galley-6b9495645d-m996v	2/2	Running	0	6m25s
istio-ingressgateway-5df799fdbd-8nqhj	1/1	Running	0	2m57s
istio-pilot-67fd786f65-nwmcb	2/2	Running	0	6m24s
istio-policy-74cf89cb66-4wrpl	2/2	Running	1	6m25s
istio-sidecar-injector-759bf6b4bc-mw4vf	1/1	Running	0	6m25s
istio-telemetry-77b6dfb4ff-zqxzz	2/2	Running	1	6m24s
istio-tracing-cd67ddf8-n4d7k	1/1	Running	0	6m25s
istiocoredns-5f7546c6f4-g7b5c	2/2	Running	0	6m39s
kiali-7964898d8c-5twln	1/1	Running	0	6m23s
prometheus-586d4445c7-xhn8d	1/1	Running	0	6m25s

`Output (do not copy)`

NAME	READY	STATUS	RESTARTS	AGE
grafana-5f798469fd-2s8k4	1/1	Running	0	59m
istio-citadel-568747d88-87kdj	1/1	Running	0	59m
istio-egressgateway-8f454cf58-zj9fs	1/1	Running	0	60m
istio-galley-6b9495645d-qfdr6	2/2	Running	0	59m
istio-ingressgateway-5df799fdbd-2c9rc	1/1	Running	0	60m
istio-pilot-67fd786f65-nzhx4	2/2	Running	0	59m
istio-policy-74cf89cb66-4bc7f	2/2	Running	3	59m
istio-sidecar-injector-759bf6b4bc-grk24	1/1	Running	0	59m
istio-telemetry-77b6dfb4ff-6zr94	2/2	Running	4	60m
istio-tracing-cd67ddf8-grs9g	1/1	Running	0	60m
istiocoredns-5f7546c6f4-gxd66	2/2	Running	0	60m
kiali-7964898d8c-nhn52	1/1	Running	0	59m
prometheus-586d4445c7-xr44v	1/1	Running	0	59m

16. 確認兩個 `dev1` 叢集都已安裝 Istio。只有 Citadel、sidecar-injector 和 coredns 會在 `dev1` 叢集中執行。這些叢集共用在 `ops-1` 叢集中執行的 Istio 控制層。

```
kubectl --context ${DEV1_GKE_1} get pods -n istio-system
kubectl --context ${DEV1_GKE_2} get pods -n istio-system
```

17. 確認兩個 `dev2` 叢集都已安裝 Istio。只有 Citadel、sidecar-injector 和 coredns 會在 `dev2` 叢集中執行。這些叢集共用在 `ops-2` 叢集中執行的 Istio 控制層。

```
kubectl --context ${DEV2_GKE_1} get pods -n istio-system
kubectl --context ${DEV2_GKE_2} get pods -n istio-system
```

`Output (do not copy)`

NAME	READY	STATUS	RESTARTS	AGE
istio-citadel-568747d88-4lj9b	1/1	Running	0	66s
istio-sidecar-injector-759bf6b4bc-ks5br	1/1	Running	0	66s
istiocoredns-5f7546c6f4-qbsqm	2/2	Running	0	78s

驗證共用控制層的服務探索

18. 視需要驗證密鑰是否已部署。

```
kubectl --context ${OPS_GKE_1} get secrets -l istio/multiCluster=true -n istio-system
kubectl --context ${OPS_GKE_2} get secrets -l istio/multiCluster=true -n istio-system
```

`Output (do not copy)`

```
For OPS_GKE_1:
NAME                                TYPE      DATA      AGE
gke-1-apps-r1a-prod                Opaque    1          8m7s
gke-2-apps-r1b-prod                Opaque    1          8m7s
gke-3-apps-r2a-prod                Opaque    1          44s
gke-4-apps-r2b-prod                Opaque    1          43s

For OPS_GKE_2:
NAME                                TYPE      DATA      AGE
gke-1-apps-r1a-prod                Opaque    1          40s
gke-2-apps-r1b-prod                Opaque    1          40s
gke-3-apps-r2a-prod                Opaque    1          8m4s
gke-4-apps-r2b-prod                Opaque    1          8m4s
```

在本研討會中，您會使用單一[共用 VPC](#)，並在其中建立所有 GKE 叢集。如要探索叢集中的服務，請使用在作業叢集中以密碼形式建立的 [kubeconfig](#) 檔案 (適用於每個應用程式叢集)。Pilot 會使用這些密鑰，透過查詢應用程式叢集的 Kube API 伺服器 (透過上述密鑰驗證)，探索服務。您會發現兩個作業叢集都能使用 kubeconfig 建立的 Secret 向所有應用程式叢集進行驗證。作業叢集可以使用 kubeconfig 檔案做為祕密方法，自動探索服務。這項作業需要作業叢集中的 Pilot 存取所有其他叢集的 Kube API 伺服器。如果 Pilot 無法連上 Kube API 伺服器，您必須手動將遠端服務新增為 [ServiceEntries](#)。您可以將 ServiceEntry 視為服務登錄中的 DNS 項目。ServiceEntry 會使用完整網域名稱 ([FQDN](#)) 和可連線的 IP 位址定義服務。詳情請參閱 [Istio 多叢集文件](#)。

6. 基礎架構存放區說明

基礎架構雲端建構

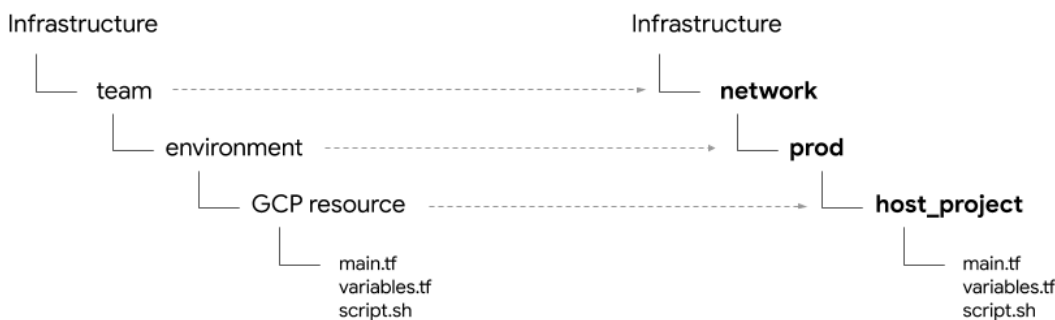
本研討會的 GCP 資源是使用 **Cloud Build** 和 **infrastructure CSR 存放區** 建構而成。您剛才從本機終端機執行了啟動程序指令碼 (位於 `scripts/bootstrap_workshop.sh`)。啟動程序指令碼會建立 GCP 資料夾、Terraform 管理員專案，以及 **Cloud Build 服務帳戶** 的適當 IAM 權限。Terraform 管理員專案用於儲存 Terraform 狀態、記錄和雜項指令碼。其中包含 **infrastructure** 和 **k8s_repo** CSR 存放區。下一節將詳細說明這些存放區。terraform 管理員專案中不會建構其他研討會資源。Terraform 管理員專案中的 Cloud Build 服務帳戶會用於建構研討會的資源。

infrastructure 資料夾中的 **cloudbuild.yaml** 檔案用於建構研討會的 GCP 資源。這會建立自訂 **建構工具映像檔**，其中包含建立 GCP 資源所需的所有工具。這些工具包括 gcloud SDK、Terraform 和其他公用程式，例如 Python、Git、jq 等。自訂建構工具映像檔會針對每個資源執行 `terraform plan` 和 `apply`。每個資源的 Terraform 檔案都位於不同的資料夾中 (詳情請參閱下一節)。資源會依一般建構順序逐一建構 (例如，先建構 GCP 專案，再於專案中建立資源)。詳情請參閱 **cloudbuild.yaml** 檔案。

只要 **infrastructure** 存放區有任何修訂版本，就會觸發 Cloud Build。對基礎架構所做的任何變更都會儲存為 **基礎架構即程式碼** (IaC)，並提交至存放區。工作坊的狀態一律會儲存在這個存放區中。

資料夾結構 - 團隊、環境和資源

基礎架構存放區會為研討會設定 GCP 基礎架構資源。並以資料夾和子資料夾的形式呈現。存放區中的基本資料夾代表擁有特定 GCP 資源的 **team**。下一層資料夾代表團隊的特定 **environment** (例如 `dev`、`stage`、`prod`)。環境中的下一層資料夾代表特定 **resource** (例如 `host_project`、`gke_clusters` 等)。資源資料夾中包含必要指令碼和 terraform 檔案。



本次研討會將介紹下列四種團隊：

1. **基礎架構** - 代表雲端基礎架構團隊。他們負責為所有其他團隊建立 GCP 資源。他們使用 Terraform 管理員專案管理資源。基礎架構存放區本身位於 Terraform 管理員專案中，Terraform 狀態檔案也是如此 (詳情請見下文)。這些資源是在啟動程序期間由 Bash 指令碼建立 (詳情請參閱「第 0 模組 - 管理員工作流程」)。
2. **network**：代表網路團隊。他們負責虛擬私有雲和網路資源，他們擁有下列 GCP 資源。
3. **host project** - 代表共用虛擬私有雲主專案。

4. `shared vpc` - 代表共用虛擬私有雲、子網路、次要 IP 範圍、路徑和防火牆規則。
5. `ops` - 代表營運/DevOps 團隊。他們擁有下列資源。
6. `ops project` - 代表所有作業資源的專案。
7. `gke clusters` - 每個區域一個作業 GKE 叢集。每個作業 GKE 叢集都會安裝 Istio 控制層。
8. `k8s-repo` - 包含所有 GKE 叢集的 GKE 資訊清單的 CSR 存放區。
9. `apps` - 代表應用程式團隊。本研討會模擬兩個團隊，分別是 `app1` 和 `app2`。他們擁有下列資源。
10. `app projects` - 每個應用程式團隊都有自己的專案集。這樣一來，他們就能控管特定專案的帳單和 IAM。
11. `gke clusters` - 這是應用程式叢集，應用程式容器/Pod 會在其中執行。
12. `gce instances` - (選用) 如果他們有在 GCE 執行個體上執行的應用程式。在本研討會中，`app1` 有幾個 GCE 執行個體，負責執行部分應用程式。

在本研討會中，應用程式 1 和應用程式 2 都以同一個應用程式 (Hipster 商店應用程式) 表示。

提供者、狀態和輸出內容 - 後端和共用狀態

`google` 和 `google-beta` 供應商位於 `gcp/[environment]/gcp/provider.tf`。每個資源資料夾中都會 [symlinked](#) `provider.tf` 檔案。這樣一來，您就能在一個位置變更供應商，不必為每個資源個別管理供應商。

每個資源都包含 `backend.tf` 檔案，用於定義資源的 `tfstate` 檔案位置。這個 `backend.tf` 檔案是使用指令碼 (位於 `scripts/setup_terraform_admin_project`) 從範本 (位於 `templates/backend.tf_tmpl`) 產生，然後放在對應的資源資料夾中。Google Cloud Storage (GCS) 值區用於後端。GCS bucket 資料夾名稱與資源名稱相符。所有資源後端都位於 Terraform 管理員專案中。

具有相互依存值的資源包含 `output.tf` 檔案。必要的輸出值會儲存在該特定資源後端定義的 `tfstate` 檔案中。舉例來說，如要在專案中建立 GKE 叢集，您必須知道專案 ID。專案 ID 會透過 `output.tf` 輸出至 `tfstate` 檔案，並可透過 GKE 叢集資源中的 `terraform_remote_state` 資料來源使用。

`shared_state` 檔案是 `terraform_remote_state` 資料來源，指向資源的 `tfstate` 檔案。資源資料夾中存在需要其他資源輸出的 `shared_state_[resource_name].tf` 檔案。舉例來說，在 `ops_gke` 資源資料夾中，有來自 `ops_project` 和 `shared_vpc` 資源的 `shared_state` 檔案，因為您需要專案 ID 和 VPC 詳細資料，才能在 `ops` 專案中建立 GKE 叢集。系統會使用指令碼 (位於 `scripts/setup_terraform_admin_project`) 從範本 (位於 `templates/shared_state.tf_tmpl`) 產生 `shared_state` 檔案。所有資源的 `shared_state` 檔案都會放在 `gcp/[environment]/shared_states` 資料夾中。系統會在對應的資源資料夾中，以符號連結方式建立必要的 `shared_state` 檔案。將所有 `shared_state` 檔案放在一個資料夾中，並將這些檔案符號連結至適當的資源資料夾，即可輕鬆在單一位置管理所有狀態檔案。

變數

所有資源值都會儲存為環境變數。這些變數會以匯出陳述式形式，儲存在 terraform 管理員專案的 GCS bucket 中，檔案名為 `vars.sh`。其中包含機構 ID、帳單帳戶、專案 ID、GKE 叢集詳細資料等。您可以從任何終端機下載並取得 `vars.sh`，以取得設定的值。

Terraform 變數會以 `TF_VAR_[variable name]` 形式儲存在 `vars.sh` 中。這些變數用於在各自的資源資料夾中產生 `variables.tfvars` 檔案。`variables.tfvars` 檔案包含所有變數及其值。`variables.tfvars` 檔案是使用指令碼 (位於 `scripts/setup_terraform_admin_project`) 從同一資料夾中的範本檔案產生。

K8s 存放區說明

`k8s_repo` 是位於 Terraform 管理員專案中的 CSR 存放區 (與基礎架構存放區分開)。用於儲存 GKE 資訊清單，並套用至所有 GKE 叢集。`k8s_repo` 是由基礎架構 Cloud Build 建立 (詳情請參閱上一節)。在初始基礎架構 Cloud Build 程序期間，系統會建立六個 GKE 叢集。在 `k8s_repo` 中，系統會建立六個資料夾。每個資料夾 (名稱與 GKE 叢集名稱相符) 都

對應一個 GKE 叢集，內含各自的資源資訊清單檔案。與建構基礎架構類似，Cloud Build 會使用 `k8s_repo`，將 Kubernetes 資訊清單套用至所有 GKE 叢集。只要 `k8s_repo` 存放區有任何修訂版本，就會觸發 Cloud Build。與基礎架構類似，所有 Kubernetes 資訊清單都會以程式碼形式儲存在 `k8s_repo` 存放區，而每個 GKE 叢集的狀態一律會儲存在各自的資料夾中。

在初始基礎架構建構過程中，系統會建立 `k8s_repo`，並在所有叢集上安裝 Istio。

專案、GKE 叢集和命名空間

本研討會的資源會劃分為不同的 GCP 專案。專案應與貴公司的機構 (或團隊) 結構相符。負責不同專案/產品/資源的團隊 (在貴機構中) 使用不同的 GCP 專案。使用不同的專案，可讓您建立多組 IAM 權限，並在專案層級管理帳單。此外，配額也是在專案層級管理。

本次研討會共有五個團隊參與，每個團隊都有自己的專案。

1. 建構 GCP 資源的**基礎架構團隊**會使用 `Terraform admin project`。他們在 CSR 存放區 (稱為 `infrastructure`) 中管理基礎架構即程式碼，並將所有與 GCP 中建構資源相關的 Terraform 狀態資訊儲存在 GCS 值區中。這些角色可管控 CSR 存放區和 Terraform 狀態 GCS 值區的存取權。
2. 建立共用虛擬私有雲的**網路團隊**會使用 `host project`。這個專案包含虛擬私有雲、子網路、路徑和防火牆規則。共用 VPC 可讓他們集中管理 GCP 資源的網路。所有專案都使用這個單一的共用 VPC 進行網路連線。
3. 建構 GKE 叢集和 ASM/Istio 控制層的**ops/platform 團隊**會使用 `ops project`。他們會管理 GKE 叢集和服務網格的壽命週期。他們負責強化叢集，並管理 Kubernetes 平台的復原能力和規模。在本研討會中，您會使用 GitOps 方法將資源部署至 Kubernetes。作業專案中存在 CSR 存放區 (稱為 `k8s_repo`)。
4. 最後，**dev1 和 dev2 團隊** (代表兩個開發團隊) 會建構應用程式，並使用自己的 `dev1` 和 `dev2 projects`。這些是您提供給顧客的應用程式和服務。這些服務建構在營運團隊管理的平台上。資源 (Deployment、Service 等) 會推送至 `k8s_repo`，並部署至適當的叢集。請注意，本研討會不會著重於 CI/CD 最佳做法和工具。您可以使用 Cloud Build，直接將 Kubernetes 資源自動部署至 GKE 叢集。在實際的正式環境情境中，您會使用適當的 CI/CD 解決方案，將應用程式部署至 GKE 叢集。

本研討會提供兩種 GKE 叢集。

1. **營運叢集** - 供營運團隊執行開發運作工具。在本研討會中，他們會執行 ASM/Istio 控制層，管理服務網格。
2. **應用程式叢集**：開發團隊用來執行應用程式。本研討會使用 Hipster 商店應用程式。

將作業/管理工具與執行應用程式的叢集分開，可讓您獨立管理每個資源的壽命週期。這兩種類型的叢集也存在於團隊/產品所屬的不同專案中，因此 IAM 權限也更容易管理。

共有六個 GKE 叢集。系統會在作業專案中建立兩個區域作業叢集。兩個作業叢集都已安裝 ASM/Istio 控制層。每個作業叢集都位於不同地區。此外，還有四個區域應用程式叢集。這些資源是在各自的專案中建立。本研討會模擬兩個開發團隊，每個團隊都有自己的專案。每個專案都包含兩個應用程式叢集。應用程式叢集是不同區域中的區域叢集。這四個應用程式叢集位於兩個區域和四個可用區。這樣就能獲得區域和可用區備援。

本研討會使用的應用程式 Hipster Shop，會部署在所有四個應用程式叢集上。每個微服務都會在每個應用程式叢集中的專屬命名空間中運作。Hipster 商店應用程式部署 (Pod) 未部署在作業叢集上。不過，所有微服務的命名空間和 Service 資源也會在作業叢集中建立。ASM/Istio 控制層會使用 Kubernetes 服務登錄檔探索服務。如果沒有服務 (在作業叢集中)，您就必須為在應用程式叢集中執行的每項服務手動建立 ServiceEntry。

在本研討會中，您將部署 10 層微服務應用程式。這個應用程式是名為「[Hipster Shop](#)」的網路電子商務應用程式，使用者可以在其中瀏覽商品、將商品加入購物車，以及購買商品。

Kubernetes 資訊清單和 k8s_repo

您可以使用 `k8s_repo`，將 Kubernetes 資源新增至所有 GKE 叢集。方法是複製 Kubernetes 資訊清單，並提交至 `k8s_repo`。所有提交都會觸發 Cloud Build 工作，將 Kubernetes 資訊清單部署至對應的叢集。`k8s_repo` 每個叢集的資

訊清單都位於與叢集名稱相同的個別資料夾中。

這六個叢集名稱分別是：

1. **gke-asm-1-r1-prod**：區域 1 中的區域作業叢集
2. **gke-asm-2-r2-prod**：區域 2 中的區域作業叢集
3. **gke-1-apps-r1a-prod**：區域 1 區域 a 中的應用程式叢集
4. **gke-2-apps-r1b-prod**：區域 1 區域 b 中的應用程式叢集
5. **gke-3-apps-r2a-prod**：區域 2 區域 a 中的應用程式叢集
6. **gke-4-apps-r2b-prod**：區域 2 區域 b 中的應用程式叢集

`k8s_repo` 會有與這些叢集對應的資料夾。放在這些資料夾中的任何資訊清單，都會套用至對應的 GKE 叢集。每個叢集的資訊清單都會放在子資料夾 (位於叢集的主資料夾內)，方便管理。在本研討會中，您將使用 [Kustomize](#) 追蹤部署的資源。詳情請參閱 Kustomize 官方說明文件。

7. 部署範例應用程式

目標：在應用程式叢集上部署 Hipster Shop 應用程式

- 複製 `k8s-repo`
- 將 Hipster 商店資訊清單複製到所有應用程式叢集
- 在作業叢集中為 Hipster shop 應用程式建立服務
- 在作業叢集中設定 `loadgenerators`，測試全域連線
- 確認與 Hipster 商店應用程式的連線安全無虞

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/deploy-sample-app.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

複製作業專案來源存放區

在初始 Terraform 基礎架構建構作業中，`k8s-repo` 已在 ops 專案中建立。

1. 為 Git 存放區建立空白目錄：

```
mkdir $WORKDIR/k8s-repo
```

2. 初始化 Git 存放區、新增遠端存放區，並從遠端存放區提取主要分支版本：

```
cd $WORKDIR/k8s-repo
git init && git remote add origin \
https://source.developers.google.com/p/$TF_VAR_ops_project_name/r/k8s-repo
```

3. 設定本機 Git 本機設定。

```
git config --local user.email $MY_USER
git config --local user.name "K8s repo user"
git config --local \
credential.'https://source.developers.google.com'.helper gcloud.sh
git pull origin master
```

複製資訊清單、提交並推送

4. 將 Hipster Shop 命名空間和服務複製到所有叢集的來源存放區。

```
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/namespaces \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app/.

cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app/services \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app/.
```

5. 將應用程式資料夾 kustomization.yaml 複製到所有叢集。

```
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app/
cp $WORKDIR/asm/k8s_manifests/prod/app/kustomization.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app/
```

6. 將 Hipster Shop Deployments、RBAC 和 PodSecurityPolicy 複製到應用程式叢集的來源存放區。

```

cp -r $WORKDIR/asm/k8s_manifests/prod/app/deployments \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/deployments \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/deployments \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/deployments \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/

cp -r $WORKDIR/asm/k8s_manifests/prod/app/rbac \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/rbac \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/rbac \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/rbac \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/

cp -r $WORKDIR/asm/k8s_manifests/prod/app/podsecuritypolicies \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/podsecuritypolicies \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/podsecuritypolicies \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/
cp -r $WORKDIR/asm/k8s_manifests/prod/app/podsecuritypolicies \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/

```

7. 從所有開發叢集 (其中一個除外) 移除 `cartservice` 部署作業、RBAC 和 `podsecuritypolicy`。Hipstershop 並非為多叢集部署而建構，因此為避免結果不一致，我們只使用一個 `cartservice`。

```

rm $WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/deployments/app-cart-service.yaml
rm $WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/podsecuritypolicies/cart-psp.yaml
rm $WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/app/rbac/cart-rbac.yaml

rm $WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/deployments/app-cart-service.yaml
rm $WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/podsecuritypolicies/cart-psp.yaml
rm $WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app/rbac/cart-rbac.yaml

rm $WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/deployments/app-cart-service.yaml
rm $WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/podsecuritypolicies/cart-psp.yaml
rm $WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/app/rbac/cart-rbac.yaml

```

8. 只在第一個開發叢集的 `kustomization.yaml` 中新增 `cartservice` 部署作業、rbac 和 `podsecuritypolicy`。

```
cd ${WORKDIR}/k8s-repo/${DEV1_GKE_1_CLUSTER}/app
cd deployments && kustomize edit add resource app-cart-service.yaml
cd ../podsecuritypolicies && kustomize edit add resource cart-psp.yaml
cd ../rbac && kustomize edit add resource cart-rbac.yaml
cd ${WORKDIR}/asm
```

9. 從作業叢集的 kustomization.yaml 中移除 podsecuritypolicies、deployments 和 rbac 目錄

```
sed -i -e '/- deployments\\/d' -e '/- podsecuritypolicies\\/d' \
-e '/- rbac\\/d' \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app/kustomization.yaml
sed -i -e '/- deployments\\/d' -e '/- podsecuritypolicies\\/d' \
-e '/- rbac\\/d' \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app/kustomization.yaml
```

10. 在 RBAC 資訊清單中取代 PROJECT_ID。

```
sed -i 's/\\${PROJECT_ID}/${TF_VAR_dev1_project_name}/g' \
$WORKDIR/k8s-repo/${DEV1_GKE_1_CLUSTER}/app/rbac/*
sed -i 's/\\${PROJECT_ID}/${TF_VAR_dev1_project_name}/g' \
$WORKDIR/k8s-repo/${DEV1_GKE_2_CLUSTER}/app/rbac/*
sed -i 's/\\${PROJECT_ID}/${TF_VAR_dev2_project_name}/g' \
$WORKDIR/k8s-repo/${DEV2_GKE_1_CLUSTER}/app/rbac/*
sed -i 's/\\${PROJECT_ID}/${TF_VAR_dev2_project_name}/g' \
$WORKDIR/k8s-repo/${DEV2_GKE_2_CLUSTER}/app/rbac/*
```

11. 將 IngressGateway 和 VirtualService 資訊清單複製到作業叢集的來源存放區。

```
cp -r $WORKDIR/asm/k8s_manifests/prod/app-ingress/* \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-ingress/
cp -r $WORKDIR/asm/k8s_manifests/prod/app-ingress/* \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-ingress/
```

12. 將 [Config Connector](#) 資源複製到每個專案的其中一個叢集。

```
cp -r $WORKDIR/asm/k8s_manifests/prod/app-cnrm/* \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-cnrm/
cp -r $WORKDIR/asm/k8s_manifests/prod/app-cnrm/* \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app-cnrm/
cp -r $WORKDIR/asm/k8s_manifests/prod/app-cnrm/* \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app-cnrm/
```

13. 在 Config Connector 資訊清單中，將 PROJECT_ID 替換為實際值。

```
sed -i 's/${PROJECT_ID}/${TF_VAR_ops_project_name}/g' \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-cnrm/*
sed -i 's/${PROJECT_ID}/${TF_VAR_dev1_project_name}/g' \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/app-cnrm/*
sed -i 's/${PROJECT_ID}/${TF_VAR_dev2_project_name}/g' \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/app-cnrm/*
```

14. 將 `loadgenerator` 資訊清單 (Deployment、PodSecurityPolicy 和 RBAC) 複製到作業叢集。Hipster 商店應用程式會透過全域 Google Cloud Load Balancer (GCLB) 公開。GCLB 會接收用戶端流量 (目的地為 `frontend`)，並將流量傳送至最接近的服務執行個體。在兩個作業叢集上放置 `loadgenerator`，可確保流量會傳送至作業叢集中執行的兩個 Istio Ingress 閘道。下節將詳細說明負載平衡。

```
cp -r $WORKDIR/asm/k8s_manifests/prod/app-loadgenerator/. \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-loadgenerator/.
cp -r $WORKDIR/asm/k8s_manifests/prod/app-loadgenerator/. \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-loadgenerator/.
```

15. 在兩個作業叢集的 `loadgenerator` 資訊清單中，將作業專案 ID 替換為實際 ID。

```
sed -i 's/OPS_PROJECT_ID/${TF_VAR_ops_project_name}/g' \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-loadgenerator/loadgenerator-deployment.yaml
sed -i 's/OPS_PROJECT_ID/${TF_VAR_ops_project_name}/g' \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-loadgenerator/loadgenerator-rbac.yaml
sed -i 's/OPS_PROJECT_ID/${TF_VAR_ops_project_name}/g' \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-loadgenerator/loadgenerator-deployment.yaml
sed -i 's/OPS_PROJECT_ID/${TF_VAR_ops_project_name}/g' \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-loadgenerator/loadgenerator-rbac.yaml
```

16. 將 `loadgenerator` 資源新增至兩個作業叢集的 `kustomization.yaml`。

```
cd $WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-loadgenerator/
kustomize edit add resource loadgenerator-psp.yaml
kustomize edit add resource loadgenerator-rbac.yaml
kustomize edit add resource loadgenerator-deployment.yaml

cd $WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-loadgenerator/
kustomize edit add resource loadgenerator-psp.yaml
kustomize edit add resource loadgenerator-rbac.yaml
kustomize edit add resource loadgenerator-deployment.yaml
```

17. 將新建的修訂版本發布到「`k8s-repo`」。

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "create app namespaces and install hipster shop"
git push --set-upstream origin master
```

18. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

驗證應用程式部署作業

19. 確認所有開發叢集中，除了購物車以外的所有應用程式命名空間中的 Pod 都處於「Running」狀態。

```
for ns in ad checkout currency email frontend payment product-catalog recommendation
shipping; do
  kubectl --context $DEV1_GKE_1 get pods -n $ns;
  kubectl --context $DEV1_GKE_2 get pods -n $ns;
  kubectl --context $DEV2_GKE_1 get pods -n $ns;
  kubectl --context $DEV2_GKE_2 get pods -n $ns;
done;
```

Output (do not copy)

NAME	READY	STATUS	RESTARTS	AGE
currencyservice-5c5b8876db-pvc6s	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
currencyservice-5c5b8876db-xlk19	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
currencyservice-5c5b8876db-zdjkg	2/2	Running	0	115s
NAME	READY	STATUS	RESTARTS	AGE
currencyservice-5c5b8876db-l748q	2/2	Running	0	82s

NAME	READY	STATUS	RESTARTS	AGE
emailservice-588467b8c8-gk92n	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
emailservice-588467b8c8-rvzk9	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
emailservice-588467b8c8-mt925	2/2	Running	0	117s
NAME	READY	STATUS	RESTARTS	AGE
emailservice-588467b8c8-klqn7	2/2	Running	0	84s

NAME	READY	STATUS	RESTARTS	AGE
frontend-64b94cf46f-kkq7d	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
frontend-64b94cf46f-lwskf	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
frontend-64b94cf46f-zz7xs	2/2	Running	0	118s
NAME	READY	STATUS	RESTARTS	AGE
frontend-64b94cf46f-2vtw5	2/2	Running	0	85s

NAME	READY	STATUS	RESTARTS	AGE
paymentservice-777f6c74f8-df8ml	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
paymentservice-777f6c74f8-bdcvg	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
paymentservice-777f6c74f8-jqf28	2/2	Running	0	117s
NAME	READY	STATUS	RESTARTS	AGE
paymentservice-777f6c74f8-95x2m	2/2	Running	0	86s

NAME	READY	STATUS	RESTARTS	AGE
productcatalogservice-786dc84f84-q5g9p	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
productcatalogservice-786dc84f84-n6lp8	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
productcatalogservice-786dc84f84-gf9xl	2/2	Running	0	119s
NAME	READY	STATUS	RESTARTS	AGE
productcatalogservice-786dc84f84-v7cbr	2/2	Running	0	86s

NAME	READY	STATUS	RESTARTS	AGE
recommendationservice-5fdf959f6b-2ltrk	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE

recommendationservice-5fdf959f6b-dqd55	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
recommendationservice-5fdf959f6b-jghcl	2/2	Running	0	119s
NAME	READY	STATUS	RESTARTS	AGE
recommendationservice-5fdf959f6b-kkspz	2/2	Running	0	87s
NAME	READY	STATUS	RESTARTS	AGE
shippingservice-7bd5f569d-qqd9n	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
shippingservice-7bd5f569d-xczg5	2/2	Running	0	13m
NAME	READY	STATUS	RESTARTS	AGE
shippingservice-7bd5f569d-wfgfr	2/2	Running	0	2m
NAME	READY	STATUS	RESTARTS	AGE
shippingservice-7bd5f569d-r6t8v	2/2	Running	0	88s

20. 確認購物車命名空間中的 Pod 僅在第一個開發叢集中處於「執行中」狀態。

```
kubectl --context $DEV1_GKE_1 get pods -n cart;
```

Output (do not copy)

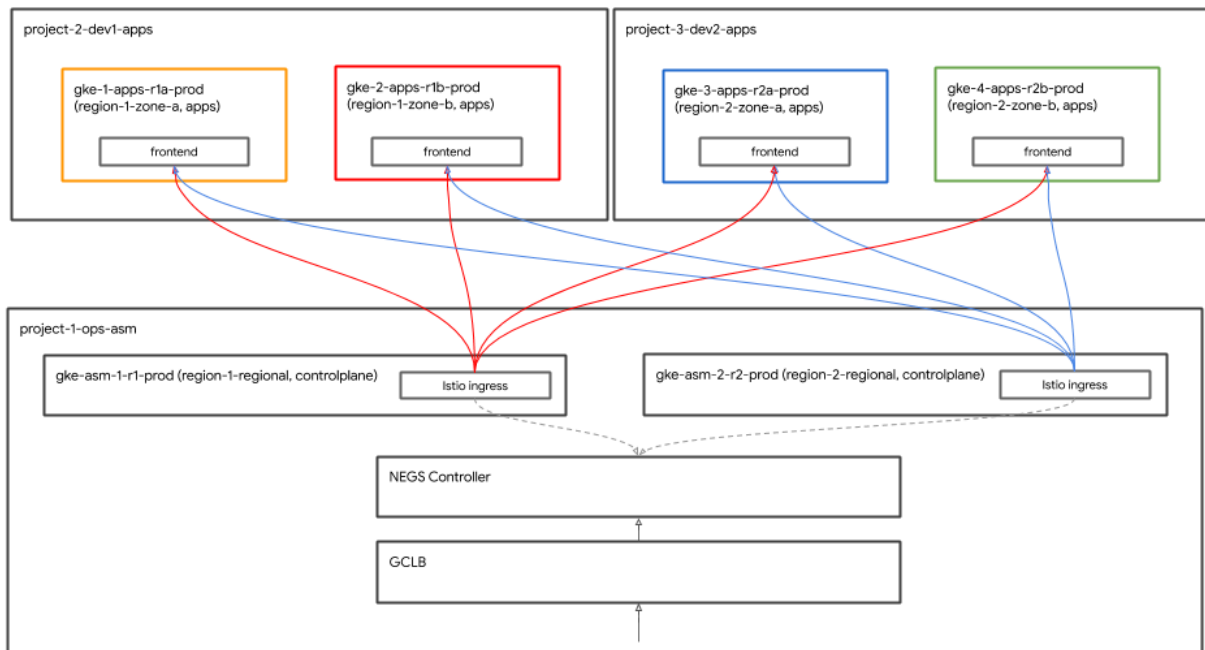
NAME	READY	STATUS	RESTARTS	AGE
cartservice-659c9749b4-vqnrd	2/2	Running	0	17m

存取 Hipster Shop 應用程式

全域負載平衡

您現在已將 Hipster Shop 應用程式部署至所有四個應用程式叢集。這些叢集位於兩個區域和四個可用區。用戶端可以存取 `frontend` 服務，進而存取 Hipster 商店應用程式。`frontend` 服務會在所有四個應用程式叢集上執行。Google Cloud 負載平衡器 ([GCLB](#)) 用於將用戶端流量導向 `frontend` 服務的所有四個執行個體。

[Istio Ingress 閘道](#) 只會在作業叢集中執行，並做為區域負載平衡器，負責區域內的兩個區域應用程式叢集。GCLB 會使用兩個 Istio Ingress 閘道 (在兩個作業叢集中執行) 做為全域前端服務的後端。Istio 輸入閘道會接收來自 GCLB 的用戶端流量，然後將用戶端流量轉送至應用程式叢集中執行的前端 Pod。



或者，您也可以直接在應用程式叢集上放置 Istio Ingress 閘道，GCLB 即可將這些閘道做為後端。

GKE Autoneg 控制器

Istio Ingress 閘道 Kubernetes 服務會使用網路端點群組 (NEG)，將自己註冊為 GCLB 的後端。NEG 可透過 GCLB 啟用容器原生負載平衡。NEG 是透過 Kubernetes 服務上的特殊註解建立，因此可以向 NEG 控制器註冊。Autoneg 控制器是特殊的 GKE 控制器，可自動建立 NEG，並使用 Service 註解將其指派為 GCLB 的後端。在初始基礎架構 Terraform Cloud Build 期間，系統會部署 Istio 控制層，包括 Istio 進入閘道。GCLB 和 autoneg 設定會在初始 Terraform 基礎架構 Cloud Build 中完成。

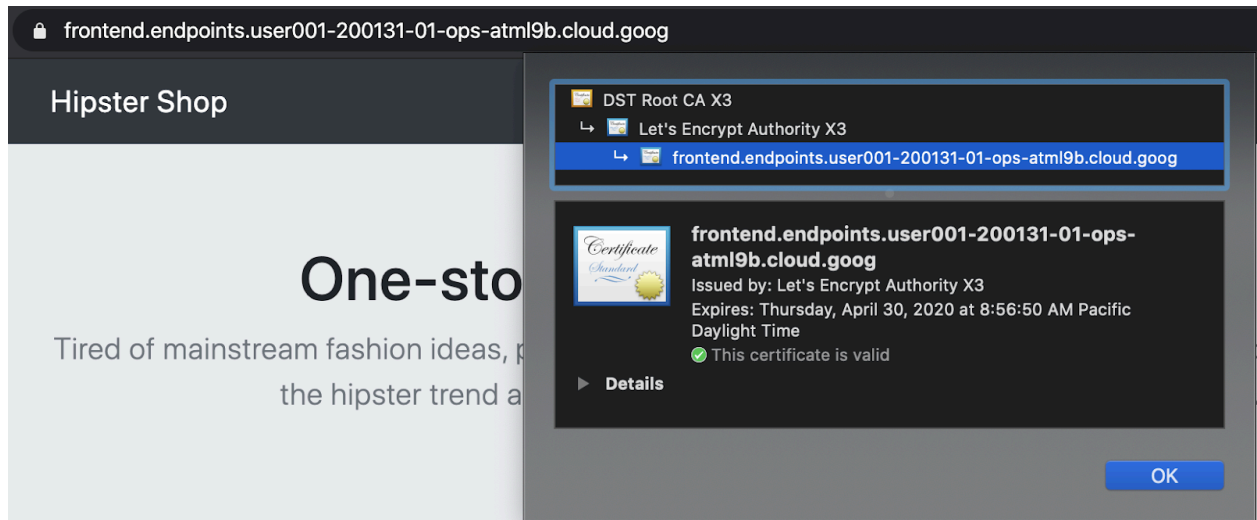
使用 Cloud Endpoints 和受管理憑證保護 Ingress 安全

GCP 管理的憑證用於保護用戶端流量，確保流量安全地傳輸至 frontend GCLB 服務。GCLB 會使用代管憑證，用於全球 frontend 服務，且憑證會在 GCLB 終止。在本研討會中，您會使用 Cloud Endpoints 做為代管憑證的網域。或者，您也可以使用網域和 frontend 的 DNS 名稱建立 GCP 管理的憑證。

21. 如要存取 Hipster 商店，請點按下列指令的連結輸出內容。

```
echo "https://frontend.endpoints.$TF_VAR_ops_project_name.cloud.goog"
```

22. 如要確認憑證是否有效，請按一下 Chrome 分頁網址列中的鎖頭符號。



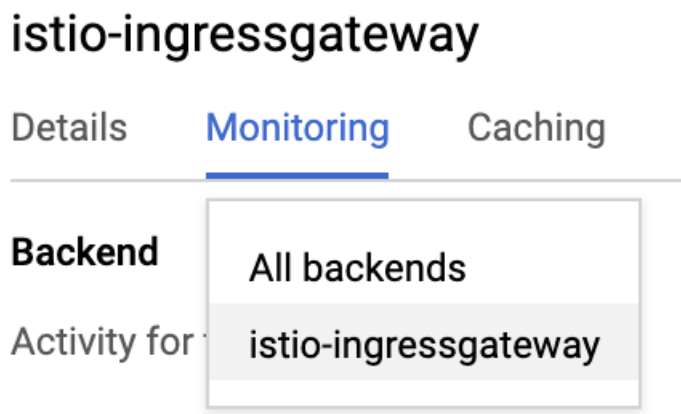
驗證全域負載平衡

在應用程式部署作業中，負載產生器會部署在兩個作業叢集中，產生測試流量，並傳送至 GCLB Hipster 商店 Cloud Endpoints 連結。確認 GCLB 正在接收流量，並將流量傳送至兩個 Istio Ingress 閘道。

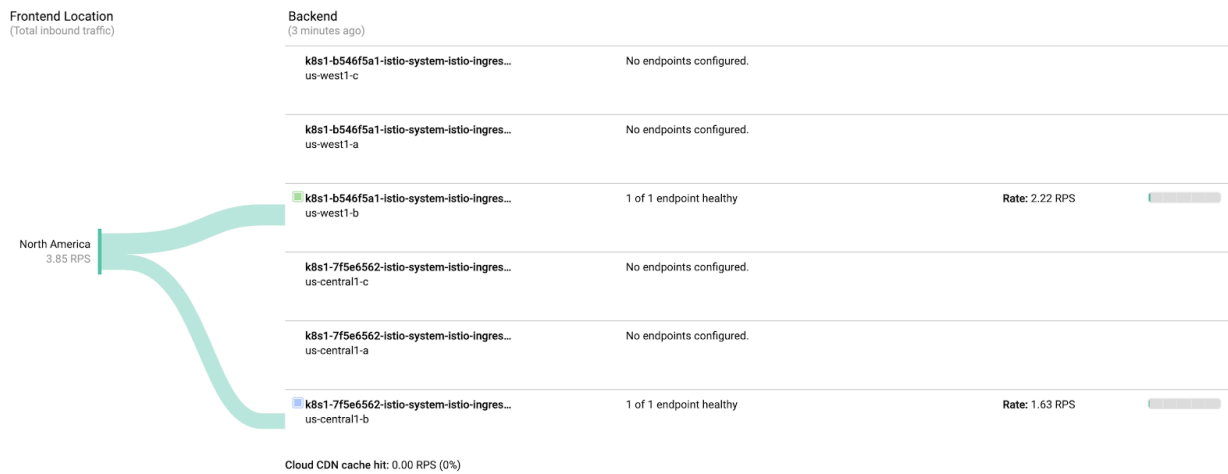
23. 取得 Hipster 商店 GCLB 建立所在作業專案的 **GCLB > Monitoring** 連結。

```
echo "https://console.cloud.google.com/net-services/loadbalancing/details/http/istio-ingressgateway?project=$TF_VAR_ops_project_name&cloudshell=false&tab=monitoring&duration=PT1H"
```

24. 如下所示，從「Backend」下拉式選單將「All backends」變更為「istio-ingressgateway」。



25. 請注意，流量會傳送至這兩個 `istio-ingressgateways`。



每個 `istio-ingressgateway` 會建立三個 NEG。由於作業叢集是地區叢集，因此系統會為該地區的每個區域建立一個 NEG。不過，`istio-ingressgateway` Pod 會在每個區域的單一可用區中執行。系統會顯示傳送至 `istio-ingressgateway` Pod 的流量。

負載產生器會在兩個作業叢集中執行，模擬來自所在區域的用戶端流量。在作業叢集區域 1 中產生的負載會傳送至區域 2 中的 `istio-ingressgateway`。同樣地，在作業叢集區域 2 中產生的負載會傳送至區域 2 中的 `istio-ingressgateway`。

8. 使用 Stackdriver 進行觀測

目標：將 Istio 遙測資料連結至 Stackdriver 並驗證。

- 安裝 `istio-telemetry` 資源
- 建立/更新 Istio 服務資訊主頁
- 查看容器記錄
- 在 Stackdriver 中查看分散式追蹤記錄

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/observability-with-stackdriver.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

Istio 的主要功能之一是內建可觀測性 (「o11y」)。也就是說，即使是黑箱、未經過儀器處理的容器，營運商仍可觀察這些容器的進出流量，為客戶提供服務。這項觀察作業會採取幾種不同的方法：指標、記錄和追蹤記錄。

我們也會使用 Hipster Shop 內建的負載產生系統。如果系統沒有流量，可觀測性就無法發揮作用，因此產生負載有助於我們瞭解可觀測性的運作方式。這項載入作業已在執行中，現在我們只能看到。

1. 安裝 `istio-to-stackdriver` 設定檔。

```
cd $WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/istio-telemetry
kustomize edit add resource istio-telemetry.yaml

cd $WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/istio-telemetry
kustomize edit add resource istio-telemetry.yaml
```

2. 提交至 `k8s-repo`。

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "Install istio to stackdriver configuration"
git push
```

3. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

4. 驗證 Istio → Stackdriver 整合功能。取得 Stackdriver 處理常式 CRD。

```
kubectl --context $OPS_GKE_1 get handler -n istio-system
```

輸出內容應顯示名為 stackdriver 的處理常式：

NAME	AGE	
kubernetesenv	12d	
prometheus	12d	
stackdriver	69s	# <== NEW!

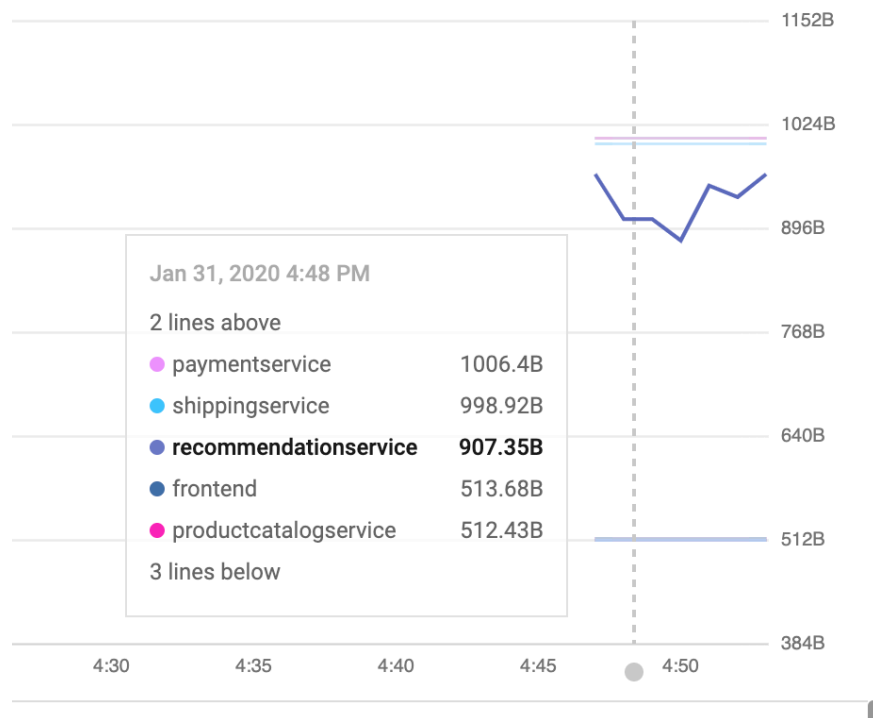
5. 確認 Istio 指標匯出至 Stackdriver 的作業正常運作。按一下這項指令輸出的連結：

```
echo "https://console.cloud.google.com/monitoring/metrics-explorer?  
cloudshell=false&project=$TF_VAR_ops_project_name"
```

系統會提示您建立以 Ops 專案命名的工作區，只要選擇「確定」即可。如果系統提示您使用新版 UI，請關閉對話方塊。

在 Metrics Explorer 的「Find resource type and metric」(尋找資源類型和指標) 下方，輸入「istio」，即可查看「Kubernetes Container」(Kubernetes 容器) 資源類型中的「Server Request Count」(伺服器要求計數) 等選項。這表示指標正從網格流入 Stackdriver。

(如要查看下方的線條，您必須依 *destination_service_name* 標籤分組)。



使用資訊主頁以視覺化方式呈現指標：

現在指標已納入 Stackdriver APM 系統，我們希望以視覺化方式呈現這些指標。在本節中，我們將安裝預先建構的資訊主頁，顯示四個「黃金信號」指標中的三個：流量(每秒要求數)、延遲時間(在本例中為第 99 個和第 50 個百分位數)，以及錯誤(本例中排除飽和度)。

Istio 的 Envoy 代理程式提供多項**指標**，但這些指標是很好的起點。(完整清單請參閱[這裡](#))。請注意，每個指標都有一組可用於篩選和匯總的標籤，例如：destination_service、source_workload_namespace、response_code、istio_tcp_received_bytes_total 等。

- 現在讓我們新增預先建立的指標**資訊主頁**。我們將直接使用 Dashboard API。您通常不會手動產生 API 呼叫，而是透過自動化系統執行，或在網頁版 UI 中手動建構資訊主頁。這樣我們就能快速開始：

```
sed -i 's/OPS_PROJECT/'${TF_VAR_ops_project_name}'/g' \
$WORKDIR/asm/k8s_manifests/prod/app-telemetry/services-dashboard.json
OAUTH_TOKEN=$(gcloud auth application-default print-access-token)
curl -X POST -H "Authorization: Bearer $OAUTH_TOKEN" -H "Content-Type: application/json" \
https://monitoring.googleapis.com/v1/projects/${TF_VAR_ops_project_name}/dashboards \
-d @$WORKDIR/asm/k8s_manifests/prod/app-telemetry/services-dashboard.json
```

- 前往下方的輸出連結，即可查看新加入的「服務資訊主頁」。

```
echo "https://console.cloud.google.com/monitoring/dashboards/custom/servicesdash?
cloudshell=false&project=${TF_VAR_ops_project_name}"
```

我們可以使用使用者體驗就地編輯資訊主頁，但以我們的案例來說，我們將使用 API 快速新增圖表。如要這麼做，請先下拉最新版資訊主頁，套用編輯內容，然後使用 HTTP PATCH 方法將其推回。

- 您可以查詢 Monitoring API，取得現有資訊主頁。取得剛新增的現有資訊主頁：

```
curl -X GET -H "Authorization: Bearer $OAUTH_TOKEN" -H "Content-Type: application/json" \
https://monitoring.googleapis.com/v1/projects/${TF_VAR_ops_project_name}/dashboards/servicesdash > /tmp/services-dashboard.json
```

- 新增圖表：(第 50 個百分位數的延遲時間)：[\[API 參考資料\]](#) 現在我們可以在程式碼中，將新的圖表小工具新增至資訊主頁。同儕可以審查這項變更，並將其納入版本控管。以下是可新增的小工具，會顯示第 50 百分位數延遲時間 (延遲時間中位數)。

請嘗試編輯剛才取得的資訊主頁，並新增節：

```
NEW_CHART=${WORKDIR}/asm/k8s_manifests/prod/app-telemetry/new-chart.json
jq --argjson newChart "$(<$NEW_CHART)" '.gridLayout.widgets += [$newChart]' /tmp/services-
dashboard.json > /tmp/patched-services-dashboard.json
```

- 更新現有服務資訊主頁：

```
curl -X PATCH -H "Authorization: Bearer $OAUTH_TOKEN" -H "Content-Type: application/json" \
https://monitoring.googleapis.com/v1/projects/$TF_VAR_ops_project_name/dashboards/servicesdash
h \
-d @/tmp/patched-services-dashboard.json
```

11. 前往下列輸出連結，查看更新後的資訊主頁：

```
echo "https://console.cloud.google.com/monitoring/dashboards/custom/servicesdash?
cloudshell=false&project=$TF_VAR_ops_project_name"
```

12. 執行簡單的記錄檔分析。

Istio 會為所有網狀網路流量提供一組**結構化記錄**，並將這些記錄上傳至 Stackdriver Logging，讓您透過單一強大工具進行跨叢集分析。記錄會註解服務層級中繼資料，例如叢集、容器、應用程式、`connection_id` 等。

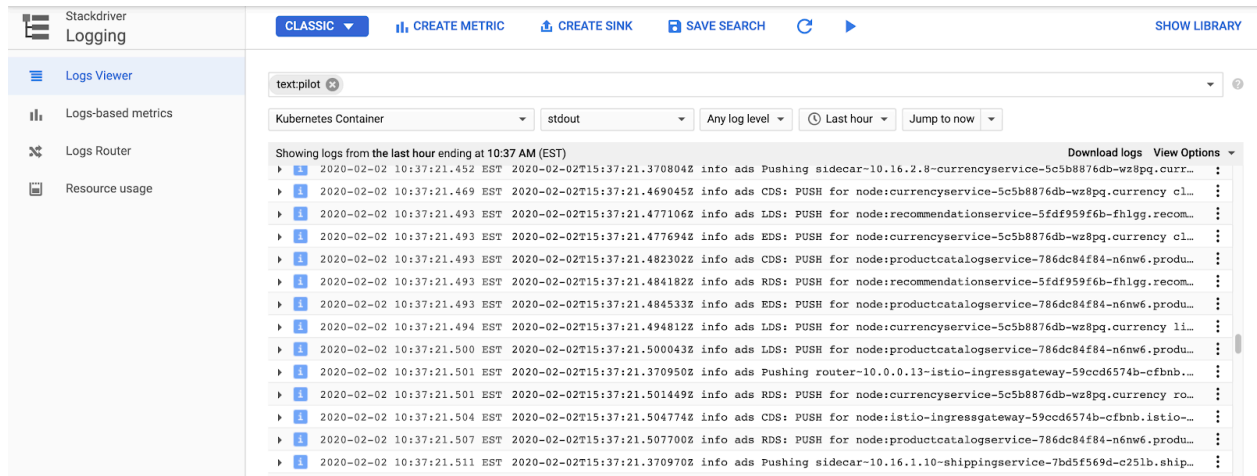
記錄項目範例 (本例為 Envoy 代理程式的 `accesslog`) 可能如下所示 (已修剪)：

```
*** DO NOT PASTE ***
logName: "projects/PROJECTNAME-11932-01-ops/logs/server-tcp-accesslog-
stackdriver.instance.istio-system"
labels: {
  connection_id: "fbb46826-96fd-476c-ac98-68a9bd6e585d-1517191"
  destination_app: "redis-cart"
  destination_ip: "10.16.1.7"
  destination_name: "redis-cart-6448dcbdcc-cj52v"
  destination_namespace: "cart"
  destination_owner: "kubernetes://apis/apps/v1/namespaces/cart/deployments/redis-cart"
  destination_workload: "redis-cart"
  source_ip: "10.16.2.8"
  total_received_bytes: "539"
  total_sent_bytes: "569"
  ...
}
```

如要查看記錄，請按照下列步驟操作：

```
echo "https://console.cloud.google.com/logs/viewer?
cloudshell=false&project=$TF_VAR_ops_project_name"
```

如要查看 Istio 控制平面記錄，請依序選取「資源」>「Kubernetes 容器」，然後搜尋「pilot」：



在這裡，我們可以看見 Istio 控制平面將 Proxy 設定推送至每個範例應用程式服務的補充 Proxy。「CDS」、「LDS」和「RDS」代表不同的 Envoy API ([詳情請參閱這篇文章](#))。

除了 Istio 記錄檔，您還可以在同一個介面中找到容器記錄檔，以及基礎架構或其他 GCP 服務記錄檔。以下列舉幾個 GKE 的 [記錄查詢範例](#)。您也可以使用記錄檢視器，根據記錄建立指標 (例如「計算符合特定字串的每個錯誤」)，並在資訊主頁上使用這些指標，或將其納入快訊。記錄也可以串流至其他分析工具，例如 BigQuery。

以下是新潮商店的篩選條件範例：

```
resource.type="k8s_container" labels.destination_app="productcatalogservice"
```

```
resource.type="k8s_container" resource.labels.namespace_name="cart"
```

13. 查看分散式追蹤記錄。

現在您使用的是分散式系統，因此需要使用新的偵錯工具：[分散式追蹤](#)。這個工具可讓您瞭解服務的互動統計資料 (例如找出下圖中異常緩慢的事件)，並深入研究原始樣本追蹤記錄，調查實際發生的詳細情況。

時間軸檢視畫面會顯示一段時間內的所有要求，並以圖表呈現延遲時間，或從初始要求到透過 Hipster 堆疊，最終回應給使用者所花費的時間。點越高，使用者體驗就越慢 (也越不開心！)。

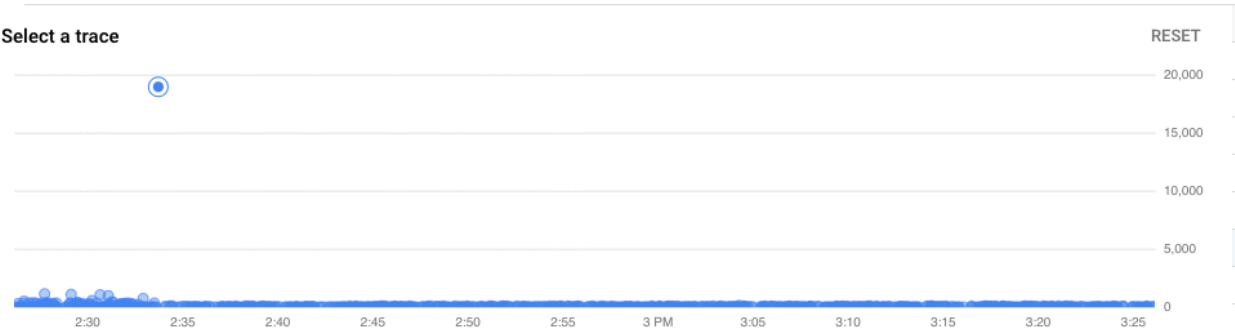
按一下點即可查看該特定要求的詳細瀑布圖。這項功能可讓您找出特定要求的原始詳細資料 (不只是匯總統計資料)，有助於瞭解服務之間的互動，特別是在找出服務之間罕見但不良的互動時。

使用過偵錯工具的人應該對瀑布圖檢視畫面不陌生，但這次顯示的不是單一應用程式不同程序所花費的時間，而是穿梭於網格中、在不同容器中執行的服務之間所花費的時間。

您可以在以下位置找到追蹤記錄：

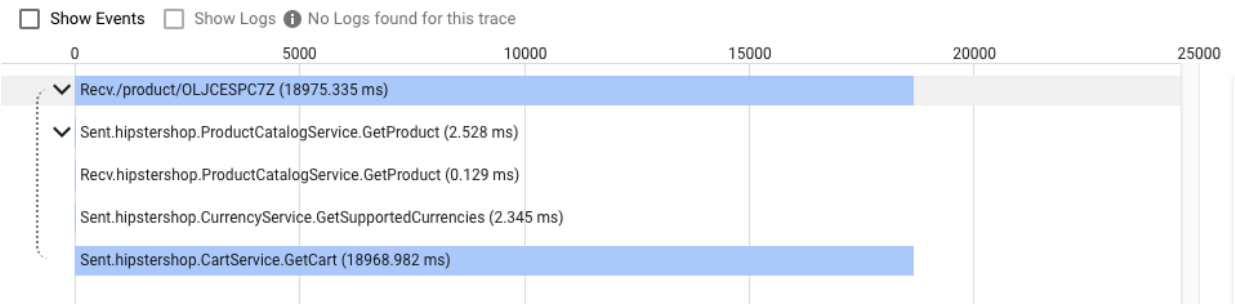
```
echo "https://console.cloud.google.com/traces/overview?cloudshell=false&project=$TF_VAR_ops_project_name"
```

工具的螢幕截圖範例如下：



[^ HIDE TRACE GRAPH](#)

Trace Waterfall View



9. 雙向傳輸層安全標準 (TLS) 驗證

目標：確保微服務之間的連線安全 (AuthN)。

- 啟用網格範圍 mTLS
- 檢查記錄，確認 mTLS 是否正常運作

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/mutual-tls.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

安裝應用程式並設定可觀測性後，我們就可以開始保護服務之間的連線，並確保連線持續運作。

舉例來說，我們可以在 Kiali 資訊主頁上看到服務並未使用 MTLS (沒有「鎖頭」圖示)。但流量正常，系統運作良好。我們的 StackDriver 黃金指標資訊主頁讓我們放心，整體而言一切運作正常。

1. 檢查作業叢集中的 MeshPolicy。注意：mTLS `PERMISSIVE` 允許加密和非 mTLS 流量。

```
kubectl --context $OPS_GKE_1 get MeshPolicy -o json | jq '.items[].spec'
kubectl --context $OPS_GKE_2 get MeshPolicy -o json | jq '.items[].spec'
```

`Output (do not copy)`

```
{
  "peers": [
    {
      "mtls": {
        "mode": "PERMISSIVE"
      }
    }
  ]
}
```

所有叢集都會使用 Istio 控制平面自訂資源 (CR)，透過 Istio 運算子設定 Istio。我們會更新 IstioControlPlane CR 並更新 k8s-repo，在所有叢集中設定 mTLS。在 IstioControlPlane CR 中將 `global > mTLS > enabled` 設為 `true`，會對 Istio 控制層造成下列兩項變更：

- MeshPolicy 設定為在所有叢集中執行的所有服務，啟用全網格 mTLS。

- 建立 DestinationRule，允許在所有叢集中執行的服務之間傳輸 ISTIO_MUTUAL 流量。

2. 我們會將 kustomize 修補程式套用至 istioControlPlane CR，以在整個叢集啟用 mTLS。將修補程式複製到所有叢集的相關目錄，並新增 kustomize 修補程式。

```
cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-replicated.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml

cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-replicated.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml

cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-shared.yaml \
$WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$DEV1_GKE_1_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml

cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-shared.yaml \
$WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$DEV1_GKE_2_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml

cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-shared.yaml \
$WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$DEV2_GKE_1_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml

cp -r $WORKDIR/asm/k8s_manifests/prod/app-mtls/mtls-kustomize-patch-shared.yaml \
$WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/istio-controlplane/mtls-kustomize-patch.yaml
cd $WORKDIR/k8s-repo/$DEV2_GKE_2_CLUSTER/istio-controlplane
kustomize edit add patch mtls-kustomize-patch.yaml
```

3. 提交至 k8s-repo。

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "turn mTLS on"
git push
```

4. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

驗證 mTLS

5. 再次檢查作業叢集中的 MeshPolicy。注意：mTLS 不再是 `PERMISSIVE`，且只允許 mTLS 流量。

```
kubectl --context $OPS_GKE_1 get MeshPolicy -o json | jq .items[].spec
kubectl --context $OPS_GKE_2 get MeshPolicy -o json | jq .items[].spec
```

輸出內容 (請勿複製)：

```
{
  "peers": [
    {
      "mtls": {}
    }
  ]
}
```

6. 說明 Istio 運算子控制器建立的 DestinationRule。

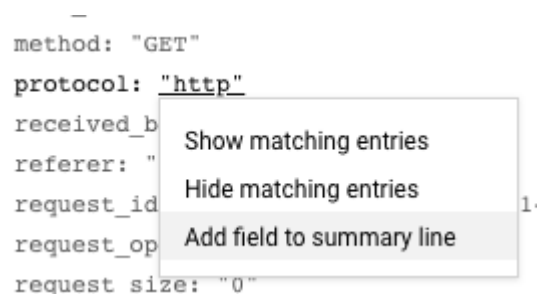
```
kubectl --context $OPS_GKE_1 get DestinationRule default -n istio-system -o json | jq '.spec'
kubectl --context $OPS_GKE_2 get DestinationRule default -n istio-system -o json | jq '.spec'
```

輸出內容 (請勿複製)：

```
{
  host: '*.local',
  trafficPolicy: {
    tls: {
      mode: ISTIO_MUTUAL
    }
  }
}
```

我們也可以在記錄中看到從 HTTP 遷移至 HTTPS 的過程。

如要從記錄檔中公開這個特定欄位，請按一下其中一個記錄項目，然後按一下要顯示的欄位值。在本例中，請按一下「protocol:」旁的「http」：



```
method: "GET"
protocol: "http"
received_b
referer: "
request_id
request_op
request size: "0"
```

The screenshot shows a log entry with a context menu open over the 'http' value in the 'protocol' field. The menu options are: 'Show matching entries', 'Hide matching entries', and 'Add field to summary line'. The 'Add field to summary line' option is highlighted. The log entry is truncated on the right with an ellipsis.

這樣就能以美觀的方式呈現轉換過程：

```
‣ i 2020-02-06 14:05:19.376 PST Total 31062 14(0.05%) 1.70
‣ i 2020-02-06 14:05:19.376 PST
‣ i 2020-02-06 14:05:19.538 PST https {"textPayload":"","insertId":"
‣ i 2020-02-06 14:05:19.544 PST Falling back to extensions/v1beta1, c
‣ i 2020-02-06 14:05:19.555 PST https {"textPayload":"","insertId":"
‣ i 2020-02-06 14:05:19.591 PST http {"textPayload":"","insertId":"8
‣ i 2020-02-06 14:05:19.591 PST http {"textPayload":"","insertId":"8
‣ i 2020-02-06 14:05:19.621 PST https {"textPayload":"","insertId":"
‣ i 2020-02-06 14:05:19.623 PST https {"textPayload":"","insertId":"
‣ i 2020-02-06 14:05:19.630 PST http {"textPayload":"","insertId":"p
‣ i 2020-02-06 14:05:19.644 PST http {"textPayload":"","insertId":"8
‣ i 2020-02-06 14:05:19.663 PST https {"textPayload":"","insertId":"
‣ i 2020-02-06 14:05:19.687 PST https {"textPayload":"","insertId":"
```

10. 初期測試部署

目標：推出前端服務的新版本。

- 在一個區域推出 `frontend-v2` (下一個正式版) 服務
- 使用 `DestinationRules` 和 `VirtualServices` 逐步將流量導向 `frontend-v2`
- 檢查提交至 `k8s-repo` 的一系列修訂版本，驗證 GitOps 部署管道

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/canary-deployments.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

初期測試部署是指逐步推出新服務。在初期測試版部署中，您會將越來越多的流量傳送至新版本，同時仍將剩餘百分比傳送至目前版本。常見模式是在流量分配的每個階段執行 **Canary 分析**，並比較新版本與基準的「黃金信號」(延遲、錯誤率、飽和度)。這有助於避免服務中斷，並確保新「v2」服務在流量分割的每個階段都能穩定運作。

在本節中，您將瞭解如何使用 Cloud Build 和 Istio 流量政策，為新版 **frontend** 服務建立基本 Canary 部署。

首先，我們會在 **DEV1 地區 (us-west1)** 中執行 Canary 管道，並在該地區的兩個叢集上推出前端 v2。其次，我們會在 **DEV2 區域 (us-central)** 中執行 Canary 管道，並將 v2 部署到該區域的兩個叢集。依序在各區域執行管道，而非在所有區域平行執行，有助於避免因設定錯誤或 v2 應用程式本身的錯誤，導致全球服務中斷。

注意：我們會在兩個區域中手動觸發 Canary 管道，但在實際運作時，您會使用自動觸發程序，例如根據推送至登錄檔的新 Docker 映像檔標記。

1. 在 Cloud Shell 中定義一些環境變數，簡化其餘指令的執行作業。

```
CANARY_DIR="$WORKDIR/asm/k8s_manifests/prod/app-canary/"  
K8S_REPO="$WORKDIR/k8s-repo"
```

2. 執行 `repo_setup.sh` 指令碼，將基準資訊清單複製到 `k8s-repo`。

```
$CANARY_DIR/repo-setup.sh
```

系統會複製下列資訊清單：

- **frontend-v2** 部署作業
- **frontend-v1** 修補程式 (包含「v1」標籤，以及具有「/version」端點的圖片)
- **respy** 是個小型 Pod，可列印 HTTP 回應分配情形，並協助我們即時呈現 Canary 部署作業。

- 前端 Istio **DestinationRule** - 根據「version」部署標籤，將前端 Kubernetes Service 分成兩個子集：v1 和 v2
- 前端 Istio **VirtualService** - 將 100% 的流量轉送至前端 v1。這會覆寫 Kubernetes Service 的預設循環配置行為，否則系統會立即將 50% 的所有 Dev1 區域流量傳送至前端 v2。

3. 將變更提交至 k8s_repo：

```
cd $K8S_REPO
git add . && git commit -am "frontend canary setup"
git push
```

4. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

5. 在 OPS1 專案的控制台中，前往 Cloud Build。等待 **Cloud Build 管道完成**，然後在兩個 DEV1 叢集的前端命名空間中取得 Pod。畫面應顯示如下：

```
watch -n 1 kubectl --context $DEV1_GKE_1 get pods -n frontend
```

Output (do not copy)

NAME	READY	STATUS	RESTARTS	AGE
frontend-578b5c5db6-h9567	2/2	Running	0	59m
frontend-v2-54b74fc75b-fbxhc	2/2	Running	0	2m26s
respy-5f4664b5f6-ff22r	2/2	Running	0	2m26s

注意：按下 Ctrl + C 鍵即可結束觀察指令

我們將使用 tmux 將 Cloud Shell 視窗分割為 2 個窗格：

- 底部窗格會執行 watch 指令，觀察前端服務的 HTTP 回應分配情形。
- 頂端窗格會執行實際的 Canary 管道指令碼。

6. 執行指令來分割 Cloud Shell 視窗，並在底部窗格中執行監看指令。


```
RESPY_POD=$(kubectl --context $DEV1_GKE_1 get pod \
-n frontend -l app=respy -o jsonpath='{..metadata.name}')
export TMUX_SESSION=$(tmux display-message -p '#S')
tmux split-window -d -t $TMUX_SESSION:0 -p33 \
-v "export KUBECONFIG=$WORKDIR/asm/gke/kubemesh; \
kubectl --context $DEV1_GKE_1 exec -n frontend -it \
$RESPY_POD -c respy /bin/sh -- -c 'watch -n 1 ./respy \
--u http://frontend:80/version --c 10 --n 500'; sleep 2"
```

輸出內容 (請勿複製)

```
500 requests to http://frontend:80/version...
+-----+-----+
| RESPONSE | % OF 500 REQUESTS |
+-----+-----+
| v1       | 100.0%             |
|          |                     |
+-----+-----+
```

我們可以發現，所有流量都會前往新 VirtualService 中定義的 frontend v1 部署作業。

7. 在 Dev1 區域執行初期測試管道。我們提供指令碼，可更新 VirtualService 中的 frontend-v2 流量百分比 (將權重更新為 20%、50%、80%，然後是 100%)。更新期間，指令碼會等待 Cloud Build 管道完成作業。針對 Dev1 區域執行初期測試部署指令碼。**注意：**這個指令碼大約需要 10 分鐘才能完成。

```
K8S_REPO=$K8S_REPO CANARY_DIR=$CANARY_DIR \
OPS_DIR=$OPS_GKE_1_CLUSTER OPS_CONTEXT=$OPS_GKE_1 \
${CANARY_DIR}/auto-canary.sh
```

在執行 respy 指令的底部視窗中，您可以即時查看流量分配情形。舉例來說，在 20% 的位置：

輸出內容 (請勿複製)

```
500 requests to http://frontend:80/version...
+-----+-----+
| RESPONSE | % OF 500 REQUESTS |
+-----+-----+
| v1       | 79.4%              |
|          |                     |
| v2       | 20.6%              |
|          |                     |
+-----+-----+
```

8. 前端 v2 的 Dev2 推出作業完成後，指令碼結尾應會顯示成功訊息：

Output (do not copy)

✅ 100% successfully deployed
🌈 frontend-v2 Canary Complete for gke-asm-1-r1-prod

9. 來自 Dev2 Pod 的所有前端流量都應傳送至 frontend-v2：

Output (do not copy)

```
500 requests to http://frontend:80/version...
+-----+-----+
| RESPONSE | % OF 500 REQUESTS |
+-----+-----+
| v2       | 100.0%             |
|          |                     |
+-----+-----+
```

10. 關閉分割窗格。

```
tmux respawn-pane -t ${TMUX_SESSION}:0.1 -k 'exit'
```

11. 前往產生的連結，開啟 Cloud Source Repos。

```
echo https://source.developers.google.com/p/${TF_VAR_ops_project_name}/r/k8s-repo
```

您應該會看到每個流量百分比都有各自的修訂版本，清單頂端則是最近的修訂版本：

✓	2c3826e	terraform	2020-01-29 19:14 -05:00	Canary: gke-asm-1-r1-prod- 100% to frontend-v2
✓	1c22487	terraform	2020-01-29 19:14 -05:00	Canary: gke-asm-1-r1-prod- 80% to frontend-v2
✓	3a30ed7	terraform	2020-01-29 19:13 -05:00	Canary: gke-asm-1-r1-prod- 50% to frontend-v2
✓	a096eda	terraform	2020-01-29 19:12 -05:00	Canary: gke-asm-1-r1-prod- 20% to frontend-v2

現在，請對 Dev2 區域重複執行相同程序。請注意，Dev2 區域在 v1 仍處於「鎖定」狀態。這是因為在基準 `repo_setup` 指令碼中，我們推送了 `VirtualService`，明確將所有流量傳送至 v1。這樣一來，我們就能在 Dev1 上安全地進行區域初期測試，並確保測試成功後，再在全球推出新版本。

12. 執行指令來分割 Cloud Shell 視窗，並在底部窗格中執行監看指令。

```
RESPY_POD=$(kubectl --context $DEV2_GKE_1 get pod \
-n frontend -l app=respy -o jsonpath='{..metadata.name}')
export TMUX_SESSION=$(tmux display-message -p '#S')
tmux split-window -d -t $TMUX_SESSION:0 -p33 \
-v "export KUBECONFIG=$WORKDIR/asm/gke/kubemesh; \
kubectl --context $DEV2_GKE_1 exec -n frontend -it \
$RESPY_POD -c respy /bin/sh -- -c 'watch -n 1 ./respy \
--u http://frontend:80/version --c 10 --n 500'; sleep 2"
```

輸出內容 (請勿複製)

```
500 requests to http://frontend:80/version...
+-----+-----+
| RESPONSE | % OF 500 REQUESTS |
+-----+-----+
| v1       | 100.0%            |
|          |                    |
+-----+-----+
```

我們可以發現，所有流量都會前往新 VirtualService 中定義的 frontend v1 部署作業。

13. 在 Dev2 區域執行初期測試管道。我們提供指令碼，可更新 VirtualService 中的 frontend-v2 流量百分比 (將權重更新為 20%、50%、80%，然後是 100%)。更新期間，指令碼會等待 Cloud Build 管道完成作業。針對 Dev1 區域執行初期測試部署指令碼。**注意：**這個指令碼大約需要 10 分鐘才能完成。

```
K8S_REPO=$K8S_REPO CANARY_DIR=$CANARY_DIR \
OPS_DIR=$OPS_GKE_2_CLUSTER OPS_CONTEXT=$OPS_GKE_2 \
${CANARY_DIR}/auto-canary.sh
```

輸出內容 (請勿複製)

```
500 requests to http://frontend:80/version...
+-----+-----+
| RESPONSE | % OF 500 REQUESTS |
+-----+-----+
| v1       | 100.0%            |
|          |                    |
+-----+-----+
```

14. 在 Dev2 的 Respy Pod 中，觀察 Dev2 Pod 的流量從前端 v1 逐步移至 v2。指令碼執行完畢後，您應該會看到：

輸出內容 (請勿複製)

```
500 requests to http://frontend:80/version...
```

+-----+-----+	
RESPONSE	% OF 500 REQUESTS
+-----+-----+	
v2	100.0%
+-----+-----+	

15. 關閉分割窗格。

```
tmux respawn-pane -t ${TMUX_SESSION}:0.1 -k 'exit'
```

本節介紹如何使用 Istio 進行區域性 Canary 部署。在實際環境中，您可能會使用觸發條件 (例如推送到容器登錄檔的新標記映像檔)，以 Cloud Build 管道的形式自動觸發這個 Canary 指令碼，而非手動執行指令碼。您也想在每個步驟之間加入 Canary 分析，根據預先定義的安全門檻分析 v2 的延遲時間和錯誤率，再傳送更多流量。

11. 授權政策

目標：在微服務之間設定 RBAC (AuthZ)。

- 建立 `AuthorizationPolicy`，拒絕存取微服務
- 建立 `AuthorizationPolicy`，允許特定微服務存取權

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/authorization-policy.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

單體式應用程式可能在單一位置執行，但全球分散式微服務應用程式會在網路界線之間進行呼叫。這表示應用程式的進入點更多，惡意攻擊的機會也更多。此外，由於 Kubernetes Pod 的 IP 是暫時性的，傳統的 IP 型防火牆規則已不足以確保工作負載之間的存取安全。在微服務架構中，需要採用新的安全防護方法。Istio 以 Kubernetes 安全性建構區塊 (例如[服務帳戶](#)) 為基礎，為應用程式提供彈性的[安全性政策](#)。

Istio 政策涵蓋驗證和授權。**驗證**會驗證身分 (這個伺服器是否符合其宣稱的身分?)，**授權**則會驗證權限 (這個用戶端是否獲准執行該操作?)。我們在第 1 模組 (MeshPolicy) 的雙向傳輸層安全標準 (mTLS) 區段中，介紹了 Istio 驗證。在本節中，我們將瞭解如何使用 Istio **授權政策**，控管對其中一個應用程式工作負載 (即 **currencyservice**) 的存取權。

首先，我們會在所有 4 個開發叢集中部署 [AuthorizationPolicy](#)，關閉所有對 `currencyservice` 的存取權，並在前端觸發錯誤。接著，我們只允許前端服務存取 `currencyservice`。

1. 檢查 `currency-deny-all.yaml` 的內容。這項政策會使用部署標籤選取器，限制對 `currencyservice` 的存取權。請注意，沒有 `spec` 欄位 - [這表示這項政策會拒絕](#)所有對所選服務的存取權。

```
cat $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-deny-all.yaml
```

輸出內容 (請勿複製)

```
apiVersion: "security.istio.io/v1beta1"
kind: "AuthorizationPolicy"
metadata:
  name: "currency-policy"
  namespace: currency
spec:
  selector:
    matchLabels:
      app: currencyservice
```

2. 將幣別政策複製到 k8s-repo，適用於兩個區域的作業叢集。

```
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-deny-all.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-authorization/currency-policy.yaml
cd $WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-authorization
kustomize edit add resource currency-policy.yaml
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-deny-all.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-authorization/currency-policy.yaml
cd $WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-authorization
kustomize edit add resource currency-policy.yaml
```

3. 推送變更。

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "AuthorizationPolicy - currency: deny all"
git push
```

4. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo https://console.cloud.google.com/cloud-build/builds?project=$TF_VAR_ops_project_name
```

5. 建構作業順利完成後，請嘗試在瀏覽器中透過下列連結存取 hipstershop 前端：

```
echo "https://frontend.endpoints.$TF_VAR_ops_project_name.cloud.goog"
```

您應該會看到來自 currencyservice 的授權錯誤訊息：

Hipster Shop

Uh, oh!

Something has failed. Below are some details for debugging.

HTTP Status: 500 Internal Server Error

```
rpc error: code = PermissionDenied desc = RBAC: access denied
failed to do currency conversion for product 9SIQT8T0J0
main.(*frontendServer).homeHandler
/go/src/github.com/GoogleCloudPlatform/microservices-demo/src/frontend/handlers.go:69
net/http.HandlerFunc.ServeHTTP
```

6. 讓我們調查貨幣服務如何強制執行這項 AuthorizationPolicy。首先，請在其中一個貨幣 Pod 的 Envoy Proxy 上啟用追蹤層級記錄，因為系統預設不會記錄遭封鎖的授權呼叫。

```
CURRENCY_POD=$(kubectl --context $DEV1_GKE_2 get pod -n currency | grep currency | awk '{
print $1 }')
kubectl --context $DEV1_GKE_2 exec -it $CURRENCY_POD -n \
currency -c istio-proxy -- curl -X POST \
"http://localhost:15000/logging?level=trace"
```

7. 從幣別服務的 Sidecar Proxy 取得 RBAC (授權) 記錄。畫面上應會顯示「enforced denied」訊息，表示 `currencyservice` 已設為封鎖所有內送要求。

```
kubectl --context $DEV1_GKE_2 logs -n currency $CURRENCY_POD \
-c istio-proxy | grep -m 3 rbac
```

輸出內容 (請勿複製)

```
[Envoy (Epoch 0)] [2020-01-30 00:45:50.815][22][debug][rbac]
[external/envoy/source/extensions/filters/http/rbac/rbac_filter.cc:67] checking
request: remoteAddress: 10.16.5.15:37310, localAddress: 10.16.3.8:7000, ssl:
uriSanPeerCertificate: spiffe://cluster.local/ns/frontend/sa/frontend,
subjectPeerCertificate: , headers: ':method', 'POST'
[Envoy (Epoch 0)] [2020-01-30 00:45:50.815][22][debug][rbac]
[external/envoy/source/extensions/filters/http/rbac/rbac_filter.cc:118] enforced
denied
[Envoy (Epoch 0)] [2020-01-30 00:45:50.815][22][debug][http]
[external/envoy/source/common/http/conn_manager_impl.cc:1354] [C115]
[S17310331589050212978] Sending local reply with details rbac_access_denied
```

8. 現在，請允許前端 (但不是其他後端服務) 存取 `currencyservice`。開啟 `currency-allow-frontend.yaml` 並檢查內容。請注意，我們新增了下列規則：

```
cat ${WORKDIR}/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend.yaml
```

輸出內容 (請勿複製)

```
rules:
- from:
  - source:
      principals: ["cluster.local/ns/frontend/sa/frontend"]
```

在這裡，我們將特定 [source.principal](#) (用戶端) 加入許可清單，允許存取幣別服務。這個 `source.principal` 是由 Kubernetes 服務帳戶定義。在本例中，我們要加入白名單的服務帳戶是前端命名空間中的前端服務帳戶。

注意：在 Istio AuthorizationPolicies 中使用 Kubernetes 服務帳戶時，您必須先啟用叢集範圍的相互 TLS，如第 1 模組所示。這是為了確保服務帳戶憑證會裝載至要求中。

9. 複製更新後的幣別政策

```
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-authorization/currency-policy.yaml
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-authorization/currency-policy.yaml
```

10. 推送變更。

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "AuthorizationPolicy - currency: allow frontend"
git push
```

11. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo https://console.cloud.google.com/cloud-build/builds?project=$TF_VAR_ops_project_name
```

12. 建構完成後，請再次開啟 Hipstershop 前端。這次首頁應該不會顯示任何錯誤，因為前端已明確獲准存取目前的服務。
13. 現在，請將商品加入購物車並點選「下單」，嘗試執行結帳。這次您應該會看到貨幣服務的價格轉換錯誤，這是因為我們只將前端加入許可清單，因此結帳服務仍無法存取貨幣服務。

Uh, oh!

Something has failed. Below are some details for debugging.

HTTP Status: 500 Internal Server Error

```
rpc error: code = Internal desc = failed to prepare order: failed to convert price of "66VCHSJNUP" to USD
failed to complete the order
main.(*frontendServer).placeOrderHandler
/go/src/github.com/GoogleCloudPlatform/microservices-demo/src/frontend/handlers.go:297
net/http.HandlerFunc.ServeHTTP
```

14. 最後，請在 currencyservice AuthorizationPolicy 中新增另一項規則，**允許結帳服務存取幣別**。請注意，我們只會開放**需要存取貨幣的兩項服務**（前端和結帳）存取貨幣。其他後端仍會遭到封鎖。
15. 開啟 `currency-allow-frontend-checkout.yaml` 並檢查內容。請注意，規則清單會以邏輯 OR 運作，因此貨幣只會接受來自這兩個服務帳戶中任一帳戶的工作負載要求。

```
cat ${WORKDIR}/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend-checkout.yaml
```

輸出內容 (請勿複製)


```
apiVersion: "security.istio.io/v1beta1"
kind: "AuthorizationPolicy"
metadata:
  name: "currency-policy"
  namespace: currency
spec:
  selector:
    matchLabels:
      app: currencyservice
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/frontend/sa/frontend"]
  - from:
    - source:
        principals: ["cluster.local/ns/checkout/sa/checkout"]
```

16. 將最終授權政策複製到 k8s-repo。

```
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend-checkout.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_1_CLUSTER/app-authorization/currency-policy.yaml
cp $WORKDIR/asm/k8s_manifests/prod/app-authorization/currency-allow-frontend-checkout.yaml \
$WORKDIR/k8s-repo/$OPS_GKE_2_CLUSTER/app-authorization/currency-policy.yaml
```

17. 推送變更

```
cd $WORKDIR/k8s-repo
git add . && git commit -am "AuthorizationPolicy - currency: allow frontend and checkout"
git push
```

18. 在先前開啟的分頁中，或點選下列連結，查看 Ops 專案 Cloud Build 的狀態：

```
echo https://console.cloud.google.com/cloud-build/builds?project=$TF_VAR_ops_project_name
```

19. 建構成功完成後，請嘗試執行結帳，應該會順利完成。

本節逐步說明如何使用 Istio 授權政策，在服務層級強制執行精細的存取權控管。在正式環境中，您可能會為每個服務建立一個 AuthorizationPolicy，並使用 [允許所有政策](#)，讓相同命名空間中的所有工作負載彼此存取。

12. 基礎架構資源調度

目標：新增區域、專案和叢集，擴充基礎架構。

- 複製 `infrastructure` 存放區
- 更新 Terraform 檔案，建立新資源
- 新區域中的 2 個子網路 (一個用於作業專案，一個用於新專案)
- 新區域中的新作業叢集 (位於新子網路中)
- 新區域的新 Istio 控制層
- 新區域中新專案的 2 個應用程式叢集
- 將新建的修訂版本發布到「`infrastructure`」存放區
- 驗證安裝

快速入門指令碼

如果選擇這個方法，請執行下列指令碼來完成本實驗室。

```
${WORKDIR}/asm/fasttrack_scripts/infrastructure-scaling.sh
```

複製及貼上方法實驗室操作說明

如果您已完成本實驗室的快速入門腳本，請勿執行下方的複製及貼上方法**。**

擴大規模的方法有很多種，如要增加運算量，可以將節點新增至現有叢集。您可以在區域中新增更多叢集。你也可以在平台上新增更多區域。要擴充平台的哪個層面，取決於需求。舉例來說，如果您在某個地區的所有三個區域都有叢集，或許只要在現有叢集中新增更多節點 (或節點集區) 即可。不過，如果您在單一區域的三個可用區中，有兩個可用區的叢集，那麼在第三個可用區新增叢集，就能擴充規模並新增容錯網域 (即新的可用區)。在區域中新增叢集的另一個原因，可能是需要建立單一租戶叢集，以符合法規或法規遵循要求 (例如 PCI，或是存放 PII 資訊的資料庫叢集)。隨著業務和服務擴展，您勢必需要新增區域，以便就近為客戶提供服務。

目前的平台包含兩個區域，每個區域有兩個可用區和叢集。您可以透過下列兩種方式擴展平台：

- **垂直：**在每個區域內新增更多運算資源。方法是在現有叢集中新增更多節點 (或節點集區)，或在區域內新增叢集。這項作業透過 `infrastructure` repo 完成。最簡單的方法是在現有叢集中新增節點。無須額外設定。新增叢集可能需要額外的子網路 (和次要範圍)、新增適當的防火牆規則、將新叢集新增至區域 ASM/Istio 服務網格控制層，以及將應用程式資源部署至新叢集。
- **水平：**新增更多區域。目前的平台提供區域範本。其中包含 ASM/Istio 控制層所在的區域作業叢集，以及部署應用程式資源的兩個 (或更多) 可用區應用程式叢集。

在本研討會中，您將「水平」擴充平台，因為平台也涵蓋垂直用途的步驟。如要透過在平台中新增區域 (r3) 的方式，水平擴充平台，必須新增下列資源：

1. 新作業和應用程式叢集的主專案共用虛擬私有雲子網路 (位於 r3 區域)。
2. 位於 r3 區域的區域作業叢集，其中包含 ASM/Istio 控制層。
3. 兩個區域應用程式叢集位於 r3 地區的兩個區域中。

4. 更新至 k8s-repo :
5. 將 ASM/Istio 控制層資源部署至 r3 區域的作業叢集。
6. 將 ASM/Istio 共用控制層資源部署至 r3 區域中的應用程式叢集。
7. 雖然您不需要建立新專案，但研討會中的步驟會示範如何新增專案 dev3，以涵蓋在平台中新增團隊的使用案例。

基礎架構存放區用於新增上述資源。

1. 在 Cloud Shell 中前往 WORKDIR，然後複製 `infrastructure` 存放區。

```
mkdir -p $WORKDIR/infra-repo
cd $WORKDIR/infra-repo
git init && git remote add origin
https://source.developers.google.com/p/${TF_ADMIN}/r/infrastructure
git config --local user.email ${MY_USER}
git config --local user.name "infra repo user"
git config --local credential.'https://source.developers.google.com'.helper gcloud.sh
git pull origin master
```

2. 將研討會來源存放區的 `add-proj` 分支版本複製到 `add-proj-repo` 目錄。

```
cd $WORKDIR
git clone https://github.com/GoogleCloudPlatform/anthos-service-mesh-workshop.git add-proj-repo
cd add-proj-repo
```

3. 從來源研討會存放區的 `add-proj` 分支版本複製檔案。 `add-proj` 分支版本包含這個部分的變更。

```
cp -r $WORKDIR/add-proj-repo/infrastructure/* $WORKDIR/infra-repo/
```

4. 將 `add-proj` 存放區目錄中的 `infrastructure` 目錄，替換為 `infra-repo` 目錄的符號連結，以便在分支上執行指令碼。

```
rm -rf $WORKDIR/add-proj-repo/infrastructure
ln -s $WORKDIR/infra-repo $WORKDIR/add-proj-repo/infrastructure
```

5. 執行 `add-project.sh` 指令碼，將共用狀態和變數複製到新的專案目錄結構。

```
$WORKDIR/add-proj-repo/scripts/add-project.sh app3 $WORKDIR/asm $WORKDIR/infra-repo
```

6. 修訂並推送變更，建立新專案

```
cd $WORKDIR/infra-repo
git add .
git status
git commit -m "add new project" && git push origin master
```

7. 提交會觸發 `infrastructure` 存放區，以部署含有新資源的基礎架構。按一下下列連結的輸出內容，然後前往頂端的最新版本，即可查看 Cloud Build 進度。

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_ADMIN}"
```

這個步驟需要 25 到 30 分鐘才能完成。現在是休息喝咖啡的好時機。

`infrastructure` Cloud Build 的最後一個步驟是在 `k8s-repo` 中建立新的 Kubernetes 資源。這會觸發 `k8s-repo` (在 ops 專案中) 的 Cloud Build。這些新的 Kubernetes 資源適用於上一個步驟中新增的三個叢集。ASM/Istio 控制層和共用控制層資源會透過 `k8s-repo` Cloud Build 新增至新叢集。

8. 基礎架構 Cloud Build 成功完成後，請按一下下列輸出連結，前往 `k8s-repo` 最新 Cloud Build 執行作業。

```
echo "https://console.cloud.google.com/cloud-build/builds?project=${TF_VAR_ops_project_name}"
```

9. 執行下列指令碼，將新叢集新增至 vars 和 kubeconfig 檔案。

```
$WORKDIR/add-proj-repo/scripts/setup-gke-vars-kubeconfig-add-proj.sh $WORKDIR/asm
```

10. 變更 `KUBECONFIG` 變數，指向新的 kubeconfig 檔案。

```
source $WORKDIR/asm/vars/vars.sh
export KUBECONFIG=$WORKDIR/asm/gke/kubemesh
```

11. 列出叢集環境。您應該會看到八個叢集。

```
kubectl config view -ojson | jq -r '.clusters[].name'
```

```
`Output (do not copy)`
```

```
gke_user001-200204-05-dev1-49tqc4_us-west1-a_gke-1-apps-r1a-prod
gke_user001-200204-05-dev1-49tqc4_us-west1-b_gke-2-apps-r1b-prod
gke_user001-200204-05-dev2-49tqc4_us-central1-a_gke-3-apps-r2a-prod
gke_user001-200204-05-dev2-49tqc4_us-central1-b_gke-4-apps-r2b-prod
gke_user001-200204-05-dev3-49tqc4_us-east1-b_gke-5-apps-r3b-prod
gke_user001-200204-05-dev3-49tqc4_us-east1-c_gke-6-apps-r3c-prod
gke_user001-200204-05-ops-49tqc4_us-central1_gke-asm-2-r2-prod
gke_user001-200204-05-ops-49tqc4_us-east1_gke-asm-3-r3-prod
gke_user001-200204-05-ops-49tqc4_us-west1_gke-asm-1-r1-prod
```

驗證 Istio 安裝

12. 確認所有 Pod 都在執行中，且作業已完成，確保 Istio 已安裝在新的作業叢集上。

```
kubectl --context $OPS_GKE_3 get pods -n istio-system
```

`Output (do not copy)`

NAME	READY	STATUS	RESTARTS	AGE
grafana-5f798469fd-72g6w	1/1	Running	0	5h12m
istio-citadel-7d8595845-hmmvj	1/1	Running	0	5h12m
istio-egressgateway-779b87c464-rw8bg	1/1	Running	0	5h12m
istio-galley-844ddfc788-zzpk1	2/2	Running	0	5h12m
istio-ingressgateway-59ccd6574b-xfj98	1/1	Running	0	5h12m
istio-pilot-7c8989f5cf-5plsg	2/2	Running	0	5h12m
istio-policy-6674bc7678-2shrk	2/2	Running	3	5h12m
istio-sidecar-injector-7795bb5888-kbl5p	1/1	Running	0	5h12m
istio-telemetry-5fd7cbbb47-c4q7b	2/2	Running	2	5h12m
istio-tracing-cd67ddf8-2qwk	1/1	Running	0	5h12m
istiocoredns-5f7546c6f4-qhj9k	2/2	Running	0	5h12m
kiali-7964898d8c-l74ww	1/1	Running	0	5h12m
prometheus-586d4445c7-x9ln6	1/1	Running	0	5h12m

13. 確認兩個 `dev3` 叢集都已安裝 Istio。只有 Citadel、sidecar-injector 和 coreDNS 會在 `dev3` 叢集中執行。這些叢集共用在 `ops-3` 叢集中執行的 Istio 控制層。

```
kubectl --context $DEV3_GKE_1 get pods -n istio-system
kubectl --context $DEV3_GKE_2 get pods -n istio-system
```

`Output (do not copy)`

NAME	READY	STATUS	RESTARTS	AGE
istio-citadel-568747d88-4lj9b	1/1	Running	0	66s
istio-sidecar-injector-759bf6b4bc-ks5br	1/1	Running	0	66s
istiocoredns-5f7546c6f4-qbsqm	2/2	Running	0	78s

驗證共用控制層的服務探索

14. 確認所有六個應用程式叢集的所有作業叢集都已部署密鑰。

```
kubectl --context $OPS_GKE_1 get secrets -l istio/multiCluster=true -n istio-system
kubectl --context $OPS_GKE_2 get secrets -l istio/multiCluster=true -n istio-system
kubectl --context $OPS_GKE_3 get secrets -l istio/multiCluster=true -n istio-system
```

`Output (do not copy)`

NAME	TYPE	DATA	AGE
gke-1-apps-r1a-prod	Opaque	1	14h
gke-2-apps-r1b-prod	Opaque	1	14h
gke-3-apps-r2a-prod	Opaque	1	14h
gke-4-apps-r2b-prod	Opaque	1	14h
gke-5-apps-r3b-prod	Opaque	1	5h12m
gke-6-apps-r3c-prod	Opaque	1	5h12m

13. 斷路機制

目標：為運送服務實作斷路器。

- 為 `shipping` 服務建立 `DestinationRule`，以實作斷路器
- 使用 `fortio` (負載產生公用程式) 強制觸發電路，驗證 `shipping` 服務的斷路器

Fast Track Script Lab Instructions

Fast Track Script Lab 即將推出！

複製及貼上方法實驗室操作說明

我們已瞭解 Istio 啟用服務的基本監控和疑難排解策略，現在來看看 Istio 如何協助您提升服務的復原能力，從一開始就減少疑難排解工作。

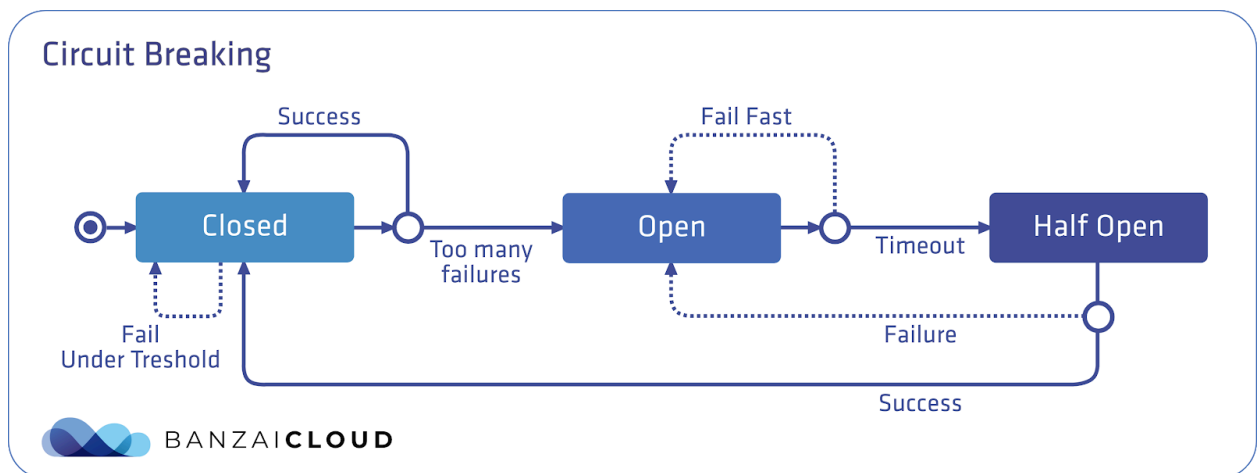
微服務架構會帶來連鎖故障的風險，也就是一項服務發生故障時，可能會影響其依附元件，以及這些依附元件的依附元件，進而造成「漣漪效應」中斷，可能影響使用者。Istio 提供斷路器流量政策，可協助您隔離服務、避免下游 (用戶端) 服務等待失敗的服務，以及在上游 (伺服器端) 服務恢復連線時，避免下游流量突然湧入。總而言之，使用斷路器可避免所有服務因某個後端服務停滯而無法達到 SLO。

斷路器模式的名稱來自於電氣開關，這種開關會在電流過大時「跳脫」，避免裝置過載。在 Istio 設定中，這表示 Envoy 是斷路器，會追蹤服務的待處理要求數量。在這個預設的關閉狀態下，要求會不間斷地流經 Envoy。

但當待處理要求數量超過您定義的門檻時，斷路器會跳脫 (開啟)，Envoy 會立即傳回錯誤。這可讓伺服器快速向用戶端回報失敗，並避免伺服器應用程式碼在超載時收到用戶端要求。

然後，在您定義的逾時時間過後，Envoy 會進入半開啟狀態，伺服器可以開始以試用方式再次接收要求，如果可以成功回應要求，斷路器就會再次關閉，要求也會再次開始傳送至伺服器。

這張圖表概略說明 Istio 斷路器模式。藍色矩形代表 Envoy，藍色實心圓圈代表用戶端，白色實心圓圈代表伺服器容器：



您可以使用 Istio DestinationRules 定義斷路器政策。在本節中，我們將套用下列政策，為運送服務強制執行斷路器：

Output (do not copy)

```
apiVersion: "networking.istio.io/v1alpha3"
kind: "DestinationRule"
metadata:
  name: "shippingservice-shipping-destrule"
  namespace: "shipping"
spec:
  host: "shippingservice.shipping.svc.cluster.local"
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutiveErrors: 1
      interval: 1s
      baseEjectionTime: 10s
      maxEjectionPercent: 100
```

請注意以下兩個 DestinationRule 欄位。 **connectionPool** 定義這項服務允許的連線數。outlierDetection 欄位用於設定 Envoy 判斷開啟斷路器門檻的方式。在這裡，Envoy 會每秒 (間隔) 計算從伺服器容器收到的錯誤數。如果超過 **consecutiveErrors** 門檻，Envoy 斷路器就會開啟，且 100% 的 productcatalog Pod 會在 10 秒內免於新的用戶端要求。Envoy 斷路器開啟 (即處於啟用狀態) 後，用戶端會收到 503 (服務無法使用) 錯誤。一起來看看實際的運作方式吧。

1. 為 k8s-repo 和 asm 目錄設定環境變數，簡化指令。

```
export K8S_REPO="${WORKDIR}/k8s-repo"
export ASM="${WORKDIR}/asm"
```

2. 更新 k8s-repo

```
cd $WORKDIR/k8s-repo
git pull
cd $WORKDIR
```

3. 在兩個 Ops 叢集上更新運送服務 DestinationRule。


```
cp $ASM/k8s_manifests/prod/istio-networking/app-shipping-circuit-breaker.yaml
${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/app-shipping-circuit-breaker.yaml
cp $ASM/k8s_manifests/prod/istio-networking/app-shipping-circuit-breaker.yaml
${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/app-shipping-circuit-breaker.yaml

cd ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/; kustomize edit add resource app-
shipping-circuit-breaker.yaml
cd ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/; kustomize edit add resource app-
shipping-circuit-breaker.yaml
```

- 將 **Fortio** 負載產生器 Pod 複製到 Dev1 地區的 GKE_1 叢集。我們會使用這個用戶端 Pod 「觸發」 shippingservice 的斷路器。

```
cp $ASM/k8s_manifests/prod/app/deployments/app-fortio.yaml
${K8S_REPO}/${DEV1_GKE_1_CLUSTER}/app/deployments/
cd ${K8S_REPO}/${DEV1_GKE_1_CLUSTER}/app/deployments; kustomize edit add resource app-
fortio.yaml
```

- 修訂變更。

```
cd $K8S_REPO
git add . && git commit -am "Circuit Breaker: shippingservice"
git push
cd $ASM
```

- 等待 Cloud Build 完成作業。
- 回到 Cloud Shell，使用 fortio Pod 將 gRPC 流量傳送至 shippingservice，並設定 1 個並行連線和 1000 個要求總數。這不會觸發斷路器，因為我們尚未超過 `connectionPool` 設定。

```
FORTIO_POD=$(kubectl --context ${DEV1_GKE_1} get pod -n shipping | grep fortio | awk '{ print $1 }')

kubectl --context ${DEV1_GKE_1} exec -it $FORTIO_POD -n shipping -c fortio /usr/bin/fortio --
load -grpc -c 1 -n 1000 -qps 0 shippingservice.shipping.svc.cluster.local:50051
```

輸出內容 (請勿複製)

```
Health SERVING : 1000
All done 1000 calls (plus 0 warmup) 4.968 ms avg, 201.2 qps
```

- 現在請再次執行 Fortio，將並行連線數增加至 2，但保持要求總數不變。我們應該會看到最多三分之二的要求傳回「溢位」錯誤，因為斷路器已觸發：在我們定義的政策中，1 秒間隔內只允許 1 個並行連線。

```
kubectl --context ${DEV1_GKE_1} exec -it $FORTIO_POD -n shipping -c fortio /usr/bin/fortio --
load -grpc -c 2 -n 1000 -qps 0 shippingservice.shipping.svc.cluster.local:50051
```

輸出內容 (請勿複製)

```
18:46:16 W grpcrunner.go:107> Error making grpc call: rpc error: code = Unavailable
desc = upstream connect error or disconnect/reset before headers. reset reason:
overflow
...
```

Health ERROR : 625

Health SERVING : 375

All done 1000 calls (plus 0 warmup) 12.118 ms avg, 96.1 qps

9. 當斷路器處於啟用狀態時，Envoy 會使用 **upstream_rq_pending_overflow** 指標追蹤捨棄的連線數量。讓我們在 fortio Pod 中找出這個項目：

```
kubectl --context ${DEV1_GKE_1} exec -it $FORTIO_POD -n shipping -c istio-proxy -- sh -c
'curl localhost:15000/stats' | grep shipping | grep pending
```

輸出內容 (請勿複製)

```
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.circuit_breakers.d
efault_rq_pending_open: 0
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.circuit_breakers.h
igh_rq_pending_open: 0
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.upstream_rq_pendin
g_active: 0
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.upstream_rq_pendin
g_failure_eject: 9
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.upstream_rq_pendin
g_overflow: 565
cluster.outbound|50051||shippingservice.shipping.svc.cluster.local.upstream_rq_pendin
g_total: 1433
```

10. 從這兩個區域移除斷路器政策，藉此清除資源。

```
kubectl --context ${OPS_GKE_1} delete destinationrule shippingservice-circuit-breaker -n shipping
rm ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/app-shipping-circuit-breaker.yaml
cd ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/; kustomize edit remove resource app-shipping-circuit-breaker.yaml
```

```
kubectl --context ${OPS_GKE_2} delete destinationrule shippingservice-circuit-breaker -n shipping
rm ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/app-shipping-circuit-breaker.yaml
cd ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/; kustomize edit remove resource app-shipping-circuit-breaker.yaml
cd ${K8S_REPO}; git add .; git commit -m "Circuit Breaker: cleanup"; git push origin master
```

本節說明如何為服務設定單一斷路器政策。最佳做法是為任何可能發生停滯的上游 (後端) 服務設定斷路器。套用 Istio 斷路器政策，有助於隔離微服務、在架構中建構容錯能力，並降低高負載下發生連鎖故障的風險。

14. 錯誤注入

目標：在建議服務推出正式版前，先導入延遲，測試服務的復原能力。

- 為 `recommendation` 服務建立 `VirtualService`，導入 5 秒延遲
- 使用 `fortio` 負載產生器測試延遲
- 移除 `VirtualService` 中的延遲並驗證

Fast Track Script Lab Instructions

Fast Track Script Lab 即將推出！

複製及貼上方法實驗室操作說明

在服務中新增斷路器政策，是建構服務彈性的一種方式，可防範實際工作環境中的服務。但斷路會導致故障 (可能發生使用者端錯誤)，這並非理想情況。為避免發生這些錯誤，並更準確預測後端傳回錯誤時，下游服務可能會如何回應，您可以在測試環境中採用**混亂測試**。混亂測試是指刻意破壞服務，藉此分析系統中的弱點，並提升容錯能力。您也可以使用混沌測試，找出後端發生故障時，如何減輕使用者遇到的錯誤，例如在前端顯示快取結果。

使用 Istio 進行錯誤注入很有幫助，因為您可以運用正式版映像檔，並在網路層新增錯誤，不必修改原始碼。在實際工作環境中，您可能會使用功能齊全的**混沌測試工具**，除了網路層之外，還能測試 Kubernetes/運算層的韌性。

您可以套用含有「fault」欄位的 `VirtualService`，使用 Istio 進行混亂測試。Istio 支援兩種故障：[延遲故障](#) (插入逾時) 和[中止故障](#) (插入 HTTP 錯誤)。在本範例中，我們將 **5 秒延遲故障** 注入**建議服務**。但這次我們不會使用斷路器針對這個閒置服務「快速失敗」，而是會強制下游服務承受完整逾時。

11. 前往錯誤注入目錄。

```
export K8S_REPO="${WORKDIR}/k8s-repo"
export ASM="${WORKDIR}/asm/"
cd $ASM
```

12. 開啟 `k8s_manifests/prod/istio-networking/app-recommendation-vs-fault.yaml` 檢查內容。請注意，Istio 可選擇將錯誤注入一定比例的要求中，這裡我們將在所有 `recommendationservice` 要求中導入逾時。

輸出內容 (請勿複製)

```

apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: recommendation-delay-fault
spec:
  hosts:
  - recommendationservice.recommendation.svc.cluster.local
  http:
  - route:
    - destination:
        host: recommendationservice.recommendation.svc.cluster.local
    fault:
      delay:
        percentage:
          value: 100
        fixedDelay: 5s

```

13. 將 VirtualService 複製到 k8s_repo。我們會在兩個區域中，於全球範圍內注入錯誤。

```

cp $ASM/k8s_manifests/prod/istio-networking/app-recommendation-vs-fault.yaml
${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/app-recommendation-vs-fault.yaml
cd ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/; kustomize edit add resource app-
recommendation-vs-fault.yaml

cp $ASM/k8s_manifests/prod/istio-networking/app-recommendation-vs-fault.yaml
${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/app-recommendation-vs-fault.yaml
cd ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/; kustomize edit add resource app-
recommendation-vs-fault.yaml

```

14. 推送變更

```

cd $K8S_REPO
git add . && git commit -am "Fault Injection: recommendationservice"
git push
cd $ASM

```

15. 等待 Cloud Build 完成作業。

16. 在斷路器部分部署的 fortio Pod 中執行，並將一些流量傳送至 recommendationservice。

```

FORTIO_POD=$(kubectl --context ${DEV1_GKE_1} get pod -n shipping | grep fortio | awk '{ print $1 }')

kubectl --context ${DEV1_GKE_1} exec -it $FORTIO_POD -n shipping -c fortio /usr/bin/fortio --
load -grpc -c 100 -n 100 -qps 0 recommendationservice.recommendation.svc.cluster.local:8080

```

Once the fortio command is complete, you should see responses averaging 5s:

輸出內容 (請勿複製)

```
Ended after 5.181367359s : 100 calls. qps=19.3
Aggregated Function Time : count 100 avg 5.0996506 +/- 0.03831 min 5.040237641 max
5.177559818 sum 509.965055
```

17. 如要查看我們注入的錯誤，請在網路瀏覽器中開啟前端，然後點選任一產品。產品頁面會擷取顯示在頁面底部的建議，因此載入時間會增加 5 秒。

18. 從兩個 Ops 叢集中移除錯誤注入服務，完成清除步驟。

```
kubectl --context ${OPS_GKE_1} delete virtualservice recommendation-delay-fault -n
recommendation
rm ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/app-recommendation-vs-fault.yaml
cd ${K8S_REPO}/${OPS_GKE_1_CLUSTER}/istio-networking/; kustomize edit remove resource app-
recommendation-vs-fault.yaml

kubectl --context ${OPS_GKE_2} delete virtualservice recommendation-delay-fault -n
recommendation
rm ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/app-recommendation-vs-fault.yaml
cd ${K8S_REPO}/${OPS_GKE_2_CLUSTER}/istio-networking/; kustomize edit remove resource app-
recommendation-vs-fault.yaml
```

19. 推送變更：

```
cd $K8S_REPO
git add . && git commit -am "Fault Injection cleanup / restore"
git push
cd $ASM
```

15. 監控 Istio 控制層

ASM 會安裝四個重要的控制層元件：Pilot、Mixer、Galley 和 Citadel。兩者都會將相關監控指標傳送至 Prometheus，而 ASM 隨附 Grafana 資訊主頁，可讓作業人員以視覺化方式呈現這項監控資料，並評估控制層的健康狀態和效能。

查看資訊主頁

20. 轉送透過 Istio 安裝的 Grafana 服務

```
kubectl --context ${OPS_GKE_1} -n istio-system port-forward svc/grafana 3000:3000 >> /dev/null
```

21. 在瀏覽器中開啟 Grafana

22. 按一下 Cloud Shell 視窗右上角的「網頁預覽」圖示

23. 按一下「Preview on port 3000」(透過以下通訊埠預覽：3000) (注意：如果通訊埠不是 3000，請按一下「change port」(變更通訊埠)，然後選取通訊埠 3000)

24. 瀏覽器會開啟分頁，網址類似於「[BASE_URL/?orgId=1&authuser=0&environment_id=default](#)」

25. 查看可用的資訊主頁

26. 將網址修改為「[BASE_URL/dashboard](#)」

27. 按一下「istio」資料夾，即可查看可用的資訊主頁

28. 點選任一資訊主頁，即可查看該元件的成效。我們會在後續章節中，探討每個元件的重要指標。

監控試用計畫

Pilot 是控制層元件，負責將網路和政策設定分配至資料層 (Envoy Proxy)。Pilot 會隨著工作負載和部署作業的數量擴充，但不一定會隨著這些工作負載的流量擴充。健康狀態不佳的 Pilot 可能會：

- 耗用超出必要的資源 (CPU 和/或 RAM)
- 導致更新的設定資訊延遲推送至 Envoy

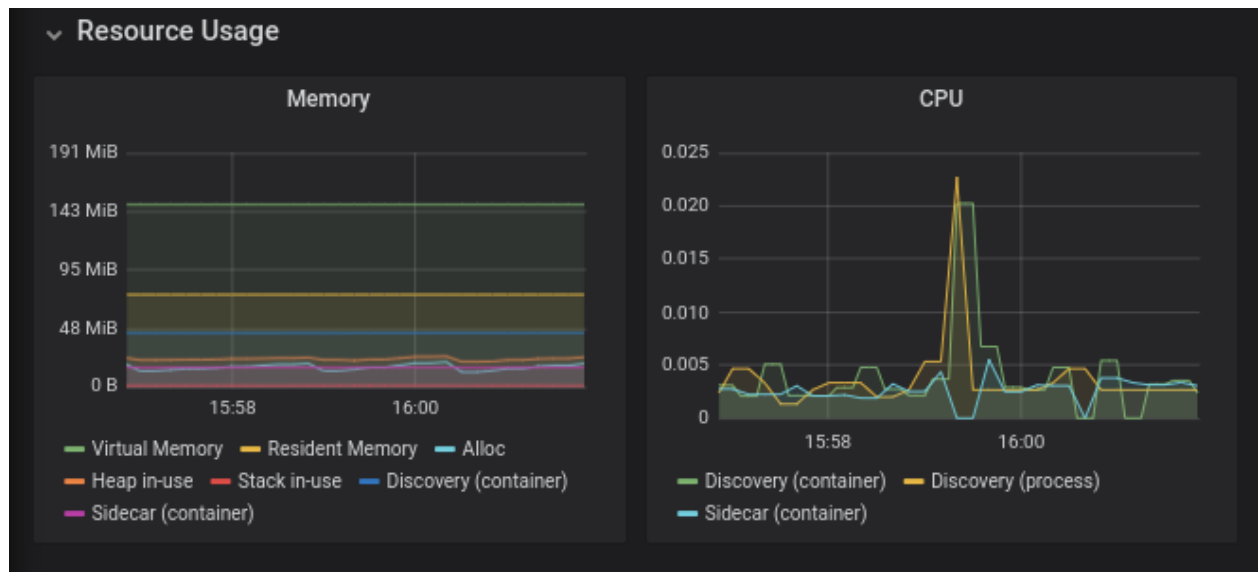
注意：如果 Pilot 發生故障或延遲，工作負載仍會提供流量。

23. 在瀏覽器中前往「[BASE_URL/dashboard/db/istio-pilot-dashboard](#)」，即可查看 Pilot 指標。

重要的監控指標

資源用量

請參閱 [Istio 效能與擴充性頁面](#)，瞭解可接受的使用量。如果持續資源用量明顯高於此值，請與 GCP 支援團隊聯絡。



前測推播資訊

這個部分會監控 Pilot 將設定推送至 Envoy Proxy 的情況。

- 「試用版推送」會顯示在任何特定時間推送的設定類型。
- **ADS Monitoring** 會顯示系統中的虛擬服務、服務和連線端點數量。
- **沒有已知端點的叢集**：顯示已設定但沒有任何執行個體的端點 (可能表示外部服務，例如 *.googleapis.com)。
- 「試播錯誤」會顯示一段時間內發生的錯誤數量。
- **衝突**：顯示監聽器上不明確的設定衝突數量。

如果發生錯誤或衝突，表示您的一或多項服務設定有誤或不一致。詳情請參閱「Troubleshooting the data plane」。

Envoy 資訊

本節包含與控制平面通訊的 Envoy Proxy 相關資訊。如果重複發生 XDS 連線失敗，請聯絡 GCP 支援團隊。

監控混音器

Mixer 會將 Envoy 代理程式的遙測資料匯集到遙測後端 (通常是 Prometheus、Stackdriver 等)。因此，它不屬於資料層。Mixer 會以兩個 Kubernetes 工作 (稱為 Mixer) 的形式部署，並使用兩個不同的服務名稱 (istio-telemetry 和 istio-policy)。

Mixer 也可用於與政策系統整合。因此，Mixer 會影響資料平面，因為 Mixer 的政策檢查失敗會封鎖服務存取權。

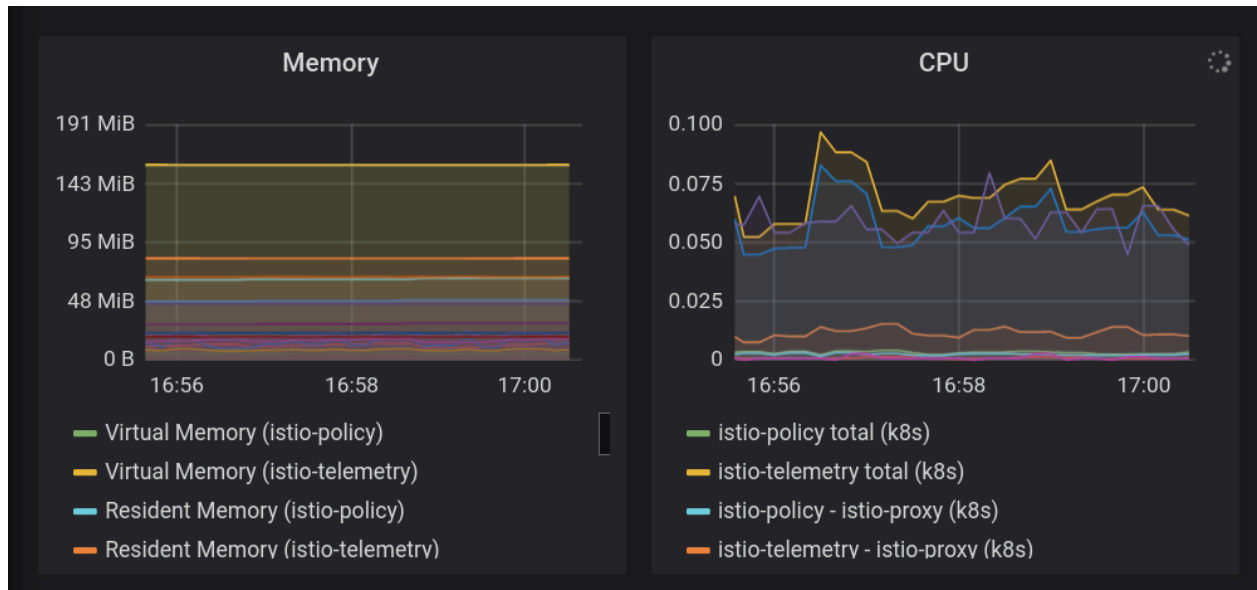
Mixer 的規模通常會隨著流量而調整。

24. 在瀏覽器中前往「[BASE_URL/dashboard/db/istio-mixer-dashboard](#)」，即可查看 Mixer 指標。

重要監控指標

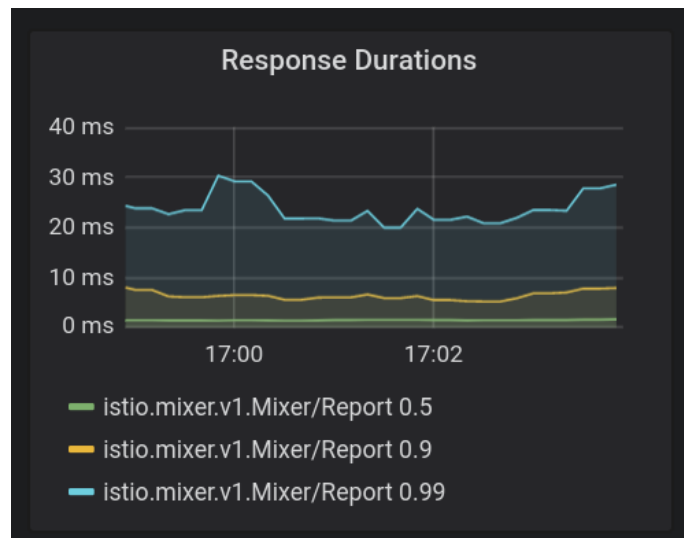
資源用量

請參閱 [Istio 效能與擴充性頁面](#)，瞭解可接受的使用量。如果持續資源用量明顯高於此值，請與 GCP 支援團隊聯絡。

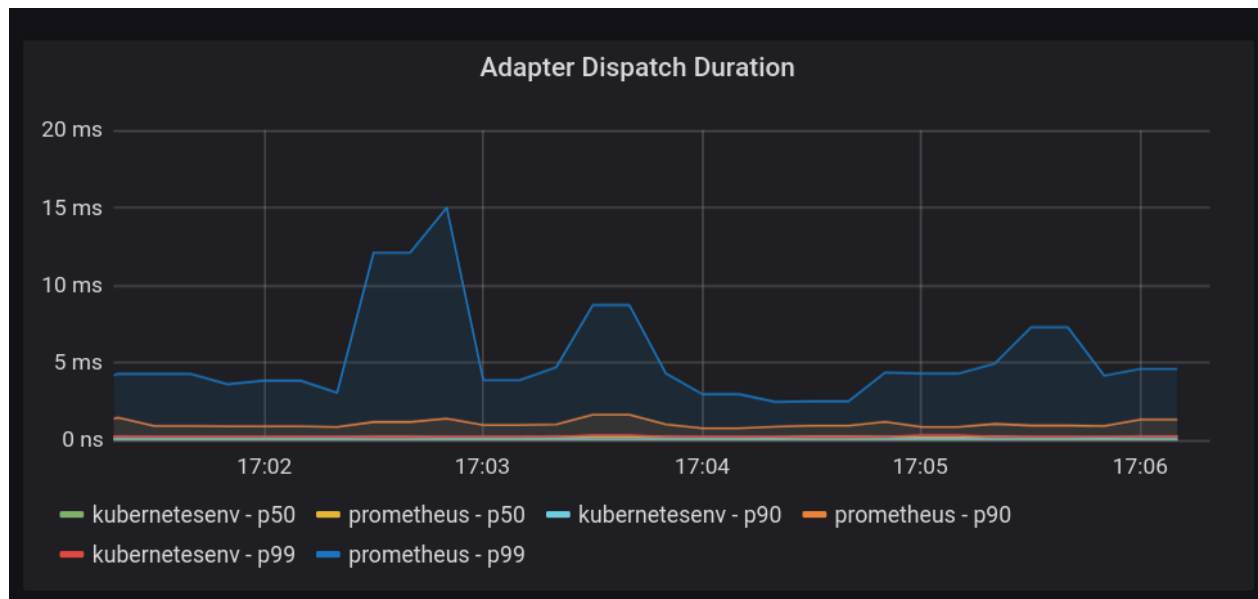


Mixer 總覽

- **回應時間**是一項重要指標。雖然 Mixer 遙測的報表不在資料路徑中，但如果這些延遲時間很長，肯定會降低 Sidecar Proxy 的效能。預期第 90 個百分位數的延遲時間為個位數毫秒，第 99 個百分位數的延遲時間則低於 100 毫秒。



- **轉接器調度時間**：表示 Mixer 在呼叫轉接器時的延遲時間 (Mixer 會透過轉接器將資訊傳送至遙測和記錄系統)。這裡的高延遲絕對會影響網格的效能。同樣地，p90 延遲時間應低於 10 毫秒。



監控校樣

Galley 是 Istio 的設定驗證、擷取、處理和發布元件。並將 Kubernetes API 伺服器的設定傳達給 Pilot。與 Pilot 類似，它會隨著系統中的服務和端點數量擴充。

25. 在瀏覽器中前往「[BASE_URL/dashboard/db/istio-galley-dashboard](#)」，即可查看 Galley 指標。

重要監控指標

資源驗證

最重要的指標是追蹤通過或未通過驗證的各種資源數量，例如目的地規則、閘道和服務項目。

連結的用戶端

指出連線至 Galley 的用戶端數量，通常為 3 個 (pilot、istio-telemetry、istio-policy)，並會隨著這些元件擴充而擴充。

16. 排解 Istio 問題

排解資料層問題

如果 Pilot 資訊主頁顯示設定有問題，您應檢查 Pilot 記錄或使用 `istioctl` 找出設定問題。

如要檢查 Pilot 記錄，請執行 `kubectl -n istio-system logs istio-pilot-69db46c598-45m44 discovery`，並將 `istio-pilot-...` 替換為要排解問題的 Pilot 執行個體 Pod ID。

在產生的記錄中，搜尋「Push Status」訊息。例如：

```

2019-11-07T01:16:20.451967Z          info          ads          Push Status: {
  "ProxyStatus": {
    "pilot_conflict_outbound_listener_tcp_over_current_tcp": {
      "0.0.0.0:443": {
        "proxy": "cartservice-7555f749f-k44dg.hipster",
        "message": "Listener=0.0.0.0:443
AcceptedTCP=accounts.google.com,*.googleapis.com RejectedTCP=edition.cnn.com TCPServices=2"
      }
    },
    "pilot_duplicate_envoy_clusters": {
      "outbound|15443|httpbin|istio-egressgateway.istio-system.svc.cluster.local": {
        "proxy": "sleep-6c66c7765d-9r85f.default",
        "message": "Duplicate cluster outbound|15443|httpbin|istio-egressgateway.istio-system.svc.cluster.local found while pushing CDS"
      },
      "outbound|443|httpbin|istio-egressgateway.istio-system.svc.cluster.local": {
        "proxy": "sleep-6c66c7765d-9r85f.default",
        "message": "Duplicate cluster outbound|443|httpbin|istio-egressgateway.istio-system.svc.cluster.local found while pushing CDS"
      },
      "outbound|80|httpbin|istio-egressgateway.istio-system.svc.cluster.local": {
        "proxy": "sleep-6c66c7765d-9r85f.default",
        "message": "Duplicate cluster outbound|80|httpbin|istio-egressgateway.istio-system.svc.cluster.local found while pushing CDS"
      }
    },
    "pilot_eds_no_instances": {
      "outbound_.80_._.frontend-external.hipster.svc.cluster.local": {},
      "outbound|443|*.googleapis.com": {},
      "outbound|443|accounts.google.com": {},
      "outbound|443|metadata.google.internal": {},
      "outbound|80|*.googleapis.com": {},
      "outbound|80|accounts.google.com": {},
      "outbound|80|frontend-external.hipster.svc.cluster.local": {},
      "outbound|80|metadata.google.internal": {}
    },
    "pilot_no_ip": {
      "loadgenerator-778c8489d6-bc65d.hipster": {
        "proxy": "loadgenerator-778c8489d6-bc65d.hipster"
      }
    }
  },
  "Version": "o1HFhx32U4s="
}

```

「推送狀態」會指出嘗試將設定推送至 Envoy 代理伺服器時發生的任何問題。在本例中，我們看到多則「重複叢集」訊息，表示上游目的地重複。

如需診斷問題的協助，請向 Google Cloud 支援團隊回報問題。

找出設定錯誤

如要使用 `istioctl` 分析設定，請執行 `istioctl experimental analyze -k --context $OPS_GKE_1`。這項工具會分析系統中的設定，指出任何問題，並提供建議的變更。如需這項指令可偵測到的完整設定錯誤清單，請參閱[說明文件](#)。

17. 清除所用資源

研討會結束後，管理員會執行下列步驟。

管理員執行 `cleanup_workshop.sh` 指令碼，刪除 `bootstrap_workshop.sh` 指令碼建立的資源。您需要提供下列資訊，才能執行清除指令碼。

- 機構名稱，例如 `yourcompany.com`
- 研討會 ID - 格式為 `YYMMDD-NN`，例如 `200131-01`
- 管理員 GCS 值區 - 在啟動程序指令碼中定義。

13. 開啟 Cloud Shell，並在 Cloud Shell 中執行下列所有動作。點選下方連結。

Cloud Shell

14. 確認您已使用預期的管理員使用者登入 `gcloud`。

```
gcloud config list
```

15. 前往 `asm` 資料夾。

```
cd ${WORKDIR}/asm
```

16. 定義要刪除的機構名稱和研討會 ID。

```
export ORGANIZATION_NAME=<ORGANIZATION NAME>
export ASM_WORKSHOP_ID=<WORKSHOP ID>
export ADMIN_STORAGE_BUCKET=<ADMIN CLOUD STORAGE BUCKET>
```

17. 執行清除指令碼，如下所示。

```
./scripts/cleanup_workshop.sh --workshop-id ${ASM_WORKSHOP_ID} --admin-gcs-bucket
${ADMIN_STORAGE_BUCKET} --org-name ${ORGANIZATION_NAME}
```